Model_3

March 24, 2026

[3]:
```python
import os
import re
import json
import time
import random
import shutil
from dataclasses import dataclass
from typing import Dict, List, Tuple, Optional

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
from sentence_transformers import SentenceTransformer


# ============================================================
# Config
# ============================================================

@dataclass
class Config:
    # Dataset
    train_subset: int = 300
    test_subset: int = 100

    # Models
    llm_name: str = "Qwen/Qwen2.5-1.5B-Instruct"
    # llm_name: str = "Qwen/Qwen2.5-3B-Instruct"
    embed_name: str = "sentence-transformers/all-MiniLM-L6-v2"

    # Oracle search
    oracle_budgets: Tuple[int, ...] = (64, 80, 96, 112, 128, 144, 160, 176,␣
 ↪192, 208, 220)
```

```
    # Supervised warm-start
    warmstart_epochs: int = 15
    warmstart_learning_rate: float = 5e-4

    # RL fine-tuning
    rl_epochs: int = 5
    policy_learning_rate: float = 3e-4
    value_learning_rate: float = 8e-4
    entropy_coef: float = 0.002
    value_loss_coef: float = 0.5
    grad_clip_norm: float = 1.0

    # Reward
    reward_token_penalty: float = 0.0001
    use_soft_reward: bool = True
    soft_reward_floor: float = 0.0

    # Thinking budget range
    min_budget: int = 64
    max_budget: int = 220

    # Final answer generation budget
    answer_max_new_tokens: int = 20

    # Generation
    do_sample: bool = False
    temperature: float = 0.0

    # RL exploration warmup
    warmup_epochs_rl: int = 2
    warmup_mix_rl: float = 0.25

    # Hardware
    seed: int = 42
    device: str = "cuda" if torch.cuda.is_available() else "cpu"

    # Logging
    print_every: int = 20
    debug_examples: int = 3
    eval_print_every: int = 10

    # Cache / outputs
    cache_dir: str = "./cache_rl_two_stage_v8"
    clear_cache_on_start: bool = True
    oracle_cache_path: str = "oracle_budget_labels_v8.json"
```

```
    # Saved models
    best_warmstart_policy_path: str = "best_warmstart_policy_v8.pt"
    best_policy_path: str = "best_policy_v8.pt"
    best_value_path: str = "best_value_v8.pt"
    final_policy_path: str = "two_stage_continuous_budget_policy_v8_final.pt"
    final_value_path: str = "two_stage_value_net_v8_final.pt"

    # Baselines
    baseline_budgets: Tuple[int, ...] = (96, 128, 160, 220)


CFG = Config()


# ============================================================
# Utilities
# ============================================================

def set_seed(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)


def ensure_dir(path: str) -> None:
    os.makedirs(path, exist_ok=True)


def safe_remove_file(path: str) -> None:
    try:
        if os.path.exists(path):
            os.remove(path)
    except Exception:
        pass


def safe_remove_dir_contents(cache_dir: str) -> None:
    """
    Best-effort cleanup of files inside cache dir.
    Does NOT fail the run if some files cannot be deleted.
    """
    if not os.path.exists(cache_dir):
        return

    for root, dirs, files in os.walk(cache_dir, topdown=False):
```

```
        for name in files:
            fpath = os.path.join(root, name)
            try:
                os.remove(fpath)
            except Exception:
                pass

        for name in dirs:
            dpath = os.path.join(root, name)
            try:
                os.rmdir(dpath)
            except Exception:
                pass


def reset_cache_dir(cache_dir: str) -> None:
    """
    Robust reset that never crashes the run.

    Strategy:
    1. If cache dir exists, try renaming it to a backup folder.
    2. Create a fresh cache dir immediately.
    3. Try deleting old backup in background-like best effort.
    4. If rename fails, best-effort clean contents only.
    """
    parent = os.path.dirname(os.path.abspath(cache_dir))
    base = os.path.basename(cache_dir)

    if not os.path.exists(cache_dir):
        os.makedirs(cache_dir, exist_ok=True)
        return

    timestamp = time.strftime("%Y%m%d_%H%M%S")
    backup_dir = os.path.join(parent, f"{base}_old_{timestamp}")

    # First preference: rename old cache folder
    try:
        os.rename(cache_dir, backup_dir)
        print(f"Renamed old cache directory to: {backup_dir}")
        os.makedirs(cache_dir, exist_ok=True)

        # Best-effort cleanup of renamed directory
        try:
            shutil.rmtree(backup_dir, ignore_errors=True)
        except Exception:
            pass
```

```
        return
    except Exception as e:
        print(f"Rename-based cache reset failed: {e}")

    # Second preference: clean only JSON/TMP cache files inside existing dir
    try:
        safe_remove_dir_contents(cache_dir)
        os.makedirs(cache_dir, exist_ok=True)
        print(f"Best-effort cleaned cache directory contents: {cache_dir}")
        return
    except Exception as e:
        print(f"Content cleanup failed: {e}")

    # Final fallback: just keep using existing directory
    os.makedirs(cache_dir, exist_ok=True)
    print(f"Proceeding with existing cache directory without full reset:␣
 ↪{cache_dir}")


def clamp_budget(x: float, min_budget: int, max_budget: int) -> int:
    return int(max(min_budget, min(max_budget, round(x))))


def denormalize_budget(x: float, min_budget: int, max_budget: int) -> float:
    return x * (max_budget - min_budget) + min_budget


def budget_to_normalized(budget: int, min_budget: int, max_budget: int) ->␣
 ↪float:
    return float(budget - min_budget) / float(max_budget - min_budget)


def normalize_scalar_list(values: List[float]) -> List[float]:
    if len(values) == 0:
        return values
    arr = np.array(values, dtype=np.float32)
    mean = float(arr.mean())
    std = float(arr.std()) + 1e-8
    return ((arr - mean) / std).tolist()


def clean_model_output(text: str) -> str:
    if text is None:
        return ""

    cleaned = text
```

```
    stop_markers = [
        "\nHuman:",
        "\nAssistant:",
        "\nUser:",
        "\nSystem:",
        "Human:",
        "Assistant:",
        "User:",
        "System:",
        "You are an AI assistant",
        "Provide a detailed answer",
        "I'll respond in English",
        "<|im_end|>",
        "<|endoftext|>",
    ]

    for marker in stop_markers:
        if marker in cleaned:
            cleaned = cleaned.split(marker)[0]

    return cleaned.strip()


# ============================================================
# GSM8K helpers
# ============================================================

def extract_gsm8k_final_answer(answer_text: str) -> str:
    if "####" in answer_text:
        return answer_text.split("####")[-1].strip()
    return answer_text.strip()


def normalize_numeric_string(text: str) -> str:
    if text is None:
        return ""

    text = text.strip()
    text = text.replace(",", "")
    text = text.replace("$", "")
    text = text.replace("%", "")
    text = text.strip()

    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if not matches:
        return text.lower().strip()
    return matches[-1]
```

```python
def parse_final_answer(output_text: str) -> str:
    text = clean_model_output(output_text)

    patterns = [
        r"FINAL ANSWER\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Final Answer\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Answer\s*:\s*(-?\d+(?:\.\d+)?)",
    ]

    for pattern in patterns:
        m = re.search(pattern, text, re.IGNORECASE)
        if m:
            return normalize_numeric_string(m.group(1))

    lines = [line.strip() for line in text.splitlines() if line.strip()]
    first_line = lines[0] if lines else text.strip()

    m = re.search(r"-?\d+(?:\.\d+)?", first_line)
    if m:
        return normalize_numeric_string(m.group(0))

    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if matches:
        return normalize_numeric_string(matches[0])

    return normalize_numeric_string(text)


def is_correct_prediction(pred: str, gold: str) -> int:
    return int(pred == gold)


def soft_numeric_match(pred: str, gold: str, floor: float = 0.0) -> float:
    try:
        p = float(pred)
        g = float(gold)
        error = abs(p - g)
        denom = max(abs(g), 1.0)
        score = 1.0 - (error / denom)
        return float(max(floor, min(1.0, score)))
    except Exception:
        return 0.0


def compute_reward(
```

```
    pred: str,
    gold: str,
    thinking_tokens_used: int,
    cfg: Config
) -> Tuple[float, float]:
    exact = float(is_correct_prediction(pred, gold))

    if cfg.use_soft_reward:
        answer_score = max(exact, soft_numeric_match(pred, gold, floor=cfg.
 ↪soft_reward_floor))
    else:
        answer_score = exact

    reward = answer_score - cfg.reward_token_penalty *␣
 ↪float(thinking_tokens_used)
    return reward, exact


# ============================================================
# Prompts
# ============================================================

def build_thinking_prompt(question: str) -> str:
    return (
        "Solve the following grade-school math problem carefully.\n"
        "Reason step by step.\n"
        "Keep the reasoning concise and relevant.\n"
        "Do not start a conversation.\n"
        "Do not add extra instructions.\n"
        "Only produce reasoning.\n\n"
        f"Question: {question}\n\n"
        "Reasoning:"
    )


def build_answer_prompt(question: str, thinking_text: str) -> str:
    return (
        "Use the reasoning below to produce the final numeric answer.\n"
        "Return only one line in this exact format:\n"
        "FINAL ANSWER: <number>\n\n"
        f"Question: {question}\n\n"
        f"Reasoning:\n{thinking_text}\n\n"
        "FINAL ANSWER:"
    )


# ============================================================
```

```
# Featurizer
# ============================================================

class QuestionFeaturizer:
    def __init__(self, embed_model_name: str, device: str):
        self.embedder = SentenceTransformer(embed_model_name, device=device)

    def handcrafted_features(self, question: str) -> np.ndarray:
        q = question.lower()
        tokens = q.split()

        length_feat = len(tokens)
        num_count = len(re.findall(r"\d+", q))
        sentence_len_chars = len(q)
        sentence_count = max(1, len(re.findall(r"[.!?]", q)))

        keywords = [
            "total", "left", "remaining", "each", "every",
            "more", "less", "after", "before", "twice",
            "half", "times", "altogether", "difference",
            "sum", "product", "cost", "price", "sold",
            "per", "average", "equal", "share", "then",
            "gave", "bought", "earned", "minutes", "hours",
            "days", "weeks", "fraction", "ratio", "percent",
            "another", "still", "now", "together"
        ]
        keyword_hits = [1.0 if kw in q else 0.0 for kw in keywords]

        plus_cue = 1.0 if any(w in q for w in ["more", "sum", "altogether",␣
↪"total", "together"]) else 0.0
        minus_cue = 1.0 if any(w in q for w in ["left", "remaining",␣
↪"difference", "less", "still"]) else 0.0
        mult_cue = 1.0 if any(w in q for w in ["each", "every", "times",␣
↪"product", "twice"]) else 0.0
        div_cue = 1.0 if any(w in q for w in ["per", "average", "equally",␣
↪"half", "share", "ratio"]) else 0.0
        multi_step_cue = 1.0 if any(w in q for w in ["then", "after", "before",␣
↪"altogether", "remaining", "now"]) else 0.0

        unit_time_cue = 1.0 if any(w in q for w in ["minutes", "hours", "days",␣
↪"weeks"]) else 0.0
        money_cue = 1.0 if "$" in q or any(w in q for w in ["dollar",␣
↪"dollars", "cost", "price"]) else 0.0
        fraction_cue = 1.0 if any(w in q for w in ["half", "fraction",␣
↪"percent", "ratio"]) else 0.0
```

```
        feats = np.array(
            [
                length_feat,
                num_count,
                sentence_len_chars,
                sentence_count,
                plus_cue,
                minus_cue,
                mult_cue,
                div_cue,
                multi_step_cue,
                unit_time_cue,
                money_cue,
                fraction_cue,
            ] + keyword_hits,
            dtype=np.float32
        )
        return feats

    def encode(self, question: str) -> np.ndarray:
        emb = self.embedder.encode(
            question,
            convert_to_numpy=True,
            normalize_embeddings=True
        ).astype(np.float32)
        hand = self.handcrafted_features(question)
        return np.concatenate([emb, hand], axis=0)


# ============================================================
# Policy + Value Networks
# ============================================================

class ContinuousBudgetPolicy(nn.Module):
    def __init__(self, input_dim: int, hidden_dim: int = 256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
        )
        self.mean_head = nn.Linear(hidden_dim, 1)
        self.log_std = nn.Parameter(torch.tensor([0.0], dtype=torch.float32))

    def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        h = self.net(x)
```

```
        mean = self.mean_head(h)
        log_std = self.log_std.expand_as(mean)
        return mean, log_std

    def sample_budget(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.
 ↪Tensor, torch.Tensor]:
        mean, log_std = self.forward(x)
        std = torch.exp(log_std)
        dist = torch.distributions.Normal(mean, std)
        raw = dist.rsample()
        normalized = torch.sigmoid(raw)
        log_prob = dist.log_prob(raw).sum(dim=-1)
        entropy = dist.entropy().sum(dim=-1)
        return normalized, log_prob, entropy

    def deterministic_budget(self, x: torch.Tensor) -> torch.Tensor:
        mean, _ = self.forward(x)
        return torch.sigmoid(mean)


class ValueNetwork(nn.Module):
    def __init__(self, input_dim: int, hidden_dim: int = 256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x).squeeze(-1)


# ============================================================
# LLM wrapper
# ============================================================

class LLMReasoner:
    def __init__(self, model_name: str, device: str):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)

        if self.tokenizer.pad_token is None:
            self.tokenizer.pad_token = self.tokenizer.eos_token

        dtype = torch.float16 if device == "cuda" else torch.float32
```

```
        self.model = AutoModelForCausalLM.from_pretrained(
            model_name,
            torch_dtype=dtype
        ).to(device)

        self.model.eval()
        self.device = device

        if hasattr(self.model, "generation_config") and self.model.
↪generation_config is not None:
            self.model.generation_config.do_sample = False
            self.model.generation_config.temperature = None
            self.model.generation_config.top_p = None
            self.model.generation_config.top_k = None

    def generate(
        self,
        prompt: str,
        max_new_tokens: int,
        do_sample: bool = False,
        temperature: float = 0.0
    ) -> Dict:
        inputs = self.tokenizer(prompt, return_tensors="pt").to(self.device)
        input_len = inputs["input_ids"].shape[1]

        gen_kwargs = {
            "max_new_tokens": max_new_tokens,
            "do_sample": do_sample,
            "pad_token_id": self.tokenizer.eos_token_id,
            "eos_token_id": self.tokenizer.eos_token_id,
        }

        if do_sample:
            gen_kwargs["temperature"] = temperature

        with torch.no_grad():
            outputs = self.model.generate(**inputs, **gen_kwargs)

        gen_ids = outputs[0][input_len:]
        text = self.tokenizer.decode(gen_ids, skip_special_tokens=True)
        text = clean_model_output(text)
        tokens_used = len(gen_ids)

        return {"text": text, "tokens_used": tokens_used}
```

```python
# ============================================================
# Caching
# ============================================================

def cache_key(stage: str, question: str, extra: str, model_name: str) -> str:
    import hashlib
    raw = f"{stage}||||{model_name}||||{question}||||{extra}"
    return hashlib.md5(raw.encode("utf-8")).hexdigest()


def load_from_cache(cache_dir: str, key: str) -> Optional[Dict]:
    path = os.path.join(cache_dir, f"{key}.json")
    if os.path.exists(path):
        try:
            with open(path, "r", encoding="utf-8") as f:
                return json.load(f)
        except Exception:
            return None
    return None


def save_to_cache(cache_dir: str, key: str, value: Dict) -> None:
    ensure_dir(cache_dir)
    path = os.path.join(cache_dir, f"{key}.json")
    tmp_path = os.path.join(cache_dir, f"{key}.tmp")

    with open(tmp_path, "w", encoding="utf-8") as f:
        json.dump(value, f, ensure_ascii=False, indent=2)

    os.replace(tmp_path, path)


# ============================================================
# Two-stage example run
# ============================================================

def run_two_stage_example(
    question: str,
    gold_answer: str,
    thinking_budget: int,
    llm: LLMReasoner,
    cfg: Config,
) -> Dict:
    ensure_dir(cfg.cache_dir)

    think_prompt = build_thinking_prompt(question)
```

```
    think_key = cache_key("thinking", question, f"budget={thinking_budget}",␣
↪cfg.llm_name)

    cached_think = load_from_cache(cfg.cache_dir, think_key)
    if cached_think is not None:
        thinking_text = clean_model_output(cached_think["text"])
        thinking_tokens_used = cached_think["tokens_used"]
    else:
        think_result = llm.generate(
            prompt=think_prompt,
            max_new_tokens=thinking_budget,
            do_sample=cfg.do_sample,
            temperature=cfg.temperature
        )
        thinking_text = clean_model_output(think_result["text"])
        thinking_tokens_used = think_result["tokens_used"]
        save_to_cache(cfg.cache_dir, think_key, {"text": thinking_text,␣
↪"tokens_used": thinking_tokens_used})

    answer_prompt = build_answer_prompt(question, thinking_text)
    answer_key = cache_key("answer", question, thinking_text, cfg.llm_name)

    cached_answer = load_from_cache(cfg.cache_dir, answer_key)
    if cached_answer is not None:
        answer_text = clean_model_output(cached_answer["text"])
        answer_tokens_used = cached_answer["tokens_used"]
    else:
        answer_result = llm.generate(
            prompt=answer_prompt,
            max_new_tokens=cfg.answer_max_new_tokens,
            do_sample=False,
            temperature=0.0
        )
        answer_text = clean_model_output(answer_result["text"])
        answer_tokens_used = answer_result["tokens_used"]
        save_to_cache(cfg.cache_dir, answer_key, {"text": answer_text,␣
↪"tokens_used": answer_tokens_used})

    pred = parse_final_answer(answer_text)
    reward, exact = compute_reward(pred, gold_answer, thinking_tokens_used, cfg)

    return {
        "prediction": pred,
        "gold": gold_answer,
        "correct": int(exact),
        "thinking_tokens_used": thinking_tokens_used,
        "answer_tokens_used": answer_tokens_used,
```

```
        "reward": reward,
        "thinking_text": thinking_text,
        "answer_text": answer_text,
        "thinking_budget": thinking_budget,
    }


# ============================================================
# Debug
# ============================================================

def print_debug_examples(
    data: List[Dict],
    llm: LLMReasoner,
    cfg: Config,
    budget: int,
    n: int = 3
) -> None:
    print("\n" + "=" * 100)
    print(f"DEBUG EXAMPLES WITH FIXED THINKING BUDGET = {budget}")
    print("=" * 100)

    for i, item in enumerate(data[:n]):
        question = item["question"]
        gold =␣
 ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        result = run_two_stage_example(question, gold, budget, llm, cfg)

        print(f"\n[Example {i+1}]")
        print("-" * 100)
        print("QUESTION:")
        print(question)
        print("\nTHINKING TEXT:")
        print(result["thinking_text"])
        print("\nANSWER TEXT:")
        print(result["answer_text"])
        print("\nPARSED PREDICTION:", result["prediction"])
        print("GOLD ANSWER      :", gold)
        print("CORRECT          :", result["correct"])
        print("THINK TOKENS     :", result["thinking_tokens_used"])
        print("REWARD           :", result["reward"])
        print("-" * 100)


# ============================================================
# Baselines and evaluation
# ============================================================
```

```python
def evaluate_fixed_budget(
    data: List[Dict],
    llm: LLMReasoner,
    cfg: Config,
    fixed_budget: int,
) -> Dict:
    corrects = []
    thinking_tokens = []
    rewards = []

    print(f"\nEvaluating fixed budget = {fixed_budget} on {len(data)} examples..")

    for idx, item in enumerate(data):
        question = item["question"]
        gold = normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        result = run_two_stage_example(question, gold, fixed_budget, llm, cfg)

        corrects.append(result["correct"])
        thinking_tokens.append(result["thinking_tokens_used"])
        rewards.append(result["reward"])

        if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == len(data):
            print(
                f"  Fixed budget {fixed_budget} | Done {idx+1}/{len(data)} | "
                f"Acc={np.mean(corrects):.4f} | AvgThinkTokens={np.mean(thinking_tokens):.2f} | "
                f"AvgReward={np.mean(rewards):.4f}"
            )

    return {
        "accuracy": float(np.mean(corrects)) if corrects else 0.0,
        "avg_thinking_tokens": float(np.mean(thinking_tokens)) if thinking_tokens else 0.0,
        "avg_reward": float(np.mean(rewards)) if rewards else 0.0,
        "budget": fixed_budget,
    }


def evaluate_policy_deterministic(
    data: List[Dict],
    featurizer: QuestionFeaturizer,
    policy: ContinuousBudgetPolicy,
    llm: LLMReasoner,
    cfg: Config,
```

```
) -> Dict:
    policy.eval()

    corrects = []
    thinking_tokens = []
    rewards = []
    budgets = []

    with torch.no_grad():
        for idx, item in enumerate(data):
            question = item["question"]
            gold =␣
↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
            feat = featurizer.encode(question)
            x = torch.tensor(feat, dtype=torch.float32, device=cfg.device).
↪unsqueeze(0)

            norm_budget = policy.deterministic_budget(x).item()
            budget = clamp_budget(
                denormalize_budget(norm_budget, cfg.min_budget, cfg.max_budget),
                cfg.min_budget,
                cfg.max_budget,
            )

            result = run_two_stage_example(question, gold, budget, llm, cfg)
            corrects.append(result["correct"])
            thinking_tokens.append(result["thinking_tokens_used"])
            rewards.append(result["reward"])
            budgets.append(budget)

            if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == len(data):
                print(
                    f"  Deterministic policy eval | Done {idx+1}/{len(data)} | "
                    f"Acc={np.mean(corrects):.4f} | AvgBudget={np.mean(budgets):
↪.2f}"
                )

    return {
        "accuracy": float(np.mean(corrects)) if corrects else 0.0,
        "avg_thinking_tokens": float(np.mean(thinking_tokens)) if␣
↪thinking_tokens else 0.0,
        "avg_reward": float(np.mean(rewards)) if rewards else 0.0,
        "avg_budget": float(np.mean(budgets)) if budgets else 0.0,
        "min_budget": float(np.min(budgets)) if budgets else 0.0,
        "max_budget": float(np.max(budgets)) if budgets else 0.0,
        "std_budget": float(np.std(budgets)) if budgets else 0.0,
    }
```

```
def evaluate_policy_sampled(
    data: List[Dict],
    featurizer: QuestionFeaturizer,
    policy: ContinuousBudgetPolicy,
    llm: LLMReasoner,
    cfg: Config,
) -> Dict:
    policy.eval()

    corrects = []
    thinking_tokens = []
    rewards = []
    budgets = []

    with torch.no_grad():
        for idx, item in enumerate(data):
            question = item["question"]
            gold = ␣
 ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
            feat = featurizer.encode(question)
            x = torch.tensor(feat, dtype=torch.float32, device=cfg.device).
 ↪unsqueeze(0)

            norm_budget, _, _ = policy.sample_budget(x)
            budget = clamp_budget(
                denormalize_budget(norm_budget.item(), cfg.min_budget, cfg.
 ↪max_budget),
                cfg.min_budget,
                cfg.max_budget,
            )

            result = run_two_stage_example(question, gold, budget, llm, cfg)
            corrects.append(result["correct"])
            thinking_tokens.append(result["thinking_tokens_used"])
            rewards.append(result["reward"])
            budgets.append(budget)

            if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == len(data):
                print(
                    f"  Sampled policy eval | Done {idx+1}/{len(data)} | "
                    f"Acc={np.mean(corrects):.4f} | AvgBudget={np.mean(budgets):
 ↪.2f}"
                )

    return {
```

```
        "accuracy": float(np.mean(corrects)) if corrects else 0.0,
        "avg_thinking_tokens": float(np.mean(thinking_tokens)) if␣
 ↪thinking_tokens else 0.0,
        "avg_reward": float(np.mean(rewards)) if rewards else 0.0,
        "avg_budget": float(np.mean(budgets)) if budgets else 0.0,
        "min_budget": float(np.min(budgets)) if budgets else 0.0,
        "max_budget": float(np.max(budgets)) if budgets else 0.0,
        "std_budget": float(np.std(budgets)) if budgets else 0.0,
    }


# ============================================================
# Oracle budget construction
# ============================================================

def build_oracle_budget_labels(
    train_data: List[Dict],
    llm: LLMReasoner,
    cfg: Config,
) -> List[Dict]:
    if os.path.exists(cfg.oracle_cache_path):
        print(f"\nLoading oracle labels from {cfg.oracle_cache_path} ...")
        with open(cfg.oracle_cache_path, "r", encoding="utf-8") as f:
            return json.load(f)

    print("\nBuilding oracle budget labels...")

    oracle_records = []

    for idx, item in enumerate(train_data):
        question = item["question"]
        gold =␣
 ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))

        best_budget = None
        best_reward = -1e9
        budget_rewards = {}

        for budget in cfg.oracle_budgets:
            result = run_two_stage_example(question, gold, budget, llm, cfg)
            reward = float(result["reward"])
            budget_rewards[str(budget)] = {
                "reward": reward,
                "correct": int(result["correct"]),
                "thinking_tokens_used": int(result["thinking_tokens_used"]),
                "prediction": result["prediction"],
            }
```

```
            if reward > best_reward:
                best_reward = reward
                best_budget = budget

        oracle_records.append(
            {
                "question": question,
                "gold_answer": gold,
                "best_budget": int(best_budget),
                "best_reward": float(best_reward),
                "budget_rewards": budget_rewards,
            }
        )

        if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) ==␣
↪len(train_data):
            print(
                f"  Oracle labels | Done {idx+1}/{len(train_data)} | "
                f"AvgBestBudget={np.mean([r['best_budget'] for r in␣
↪oracle_records]):.2f} | "
                f"AvgBestReward={np.mean([r['best_reward'] for r in␣
↪oracle_records]):.4f}"
            )

    with open(cfg.oracle_cache_path, "w", encoding="utf-8") as f:
        json.dump(oracle_records, f, ensure_ascii=False, indent=2)

    print(f"Saved oracle labels to {cfg.oracle_cache_path}")
    return oracle_records


# ============================================================
# Supervised warm-start
# ============================================================

def warmstart_policy_from_oracle(
    oracle_records: List[Dict],
    test_data: List[Dict],
    featurizer: QuestionFeaturizer,
    policy: ContinuousBudgetPolicy,
    llm: LLMReasoner,
    cfg: Config,
) -> None:
    print("\nStarting supervised warm-start from oracle budgets...")

    optimizer = optim.Adam(policy.parameters(), lr=cfg.warmstart_learning_rate)
```

```
    mse_loss = nn.MSELoss()

    best_eval_reward = -1e9

    for epoch in range(cfg.warmstart_epochs):
        policy.train()
        random.shuffle(oracle_records)

        losses = []
        pred_budgets = []
        target_budgets = []

        for step_idx, record in enumerate(oracle_records):
            question = record["question"]
            target_budget = int(record["best_budget"])

            feat = featurizer.encode(question)
            x = torch.tensor(feat, dtype=torch.float32, device=cfg.device).
↪unsqueeze(0)

            pred_norm = policy.deterministic_budget(x)
            target_norm = torch.tensor(
                [[budget_to_normalized(target_budget, cfg.min_budget, cfg.
↪max_budget)]],
                dtype=torch.float32,
                device=cfg.device,
            )

            loss = mse_loss(pred_norm, target_norm)

            optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(policy.parameters(), cfg.
↪grad_clip_norm)
            optimizer.step()

            losses.append(float(loss.item()))

            pred_budget = clamp_budget(
                denormalize_budget(pred_norm.item(), cfg.min_budget, cfg.
↪max_budget),
                cfg.min_budget,
                cfg.max_budget,
            )
            pred_budgets.append(pred_budget)
            target_budgets.append(target_budget)
```

```
            if (step_idx + 1) % cfg.print_every == 0 or (step_idx + 1) ==␣
↪len(oracle_records):
                mae = np.mean(np.abs(np.array(pred_budgets) - np.
↪array(target_budgets)))
                print(
                    f"[Warmstart {epoch+1}/{cfg.warmstart_epochs}] Step␣
↪{step_idx+1}/{len(oracle_records)} | "
                    f"Loss={np.mean(losses):.6f} | PredAvgBudget={np.
↪mean(pred_budgets):.2f} | "
                    f"TargetAvgBudget={np.mean(target_budgets):.2f} | MAE={mae:.
↪2f}"
                )

        print(
            f"Warmstart epoch {epoch+1} summary | "
            f"Loss={np.mean(losses):.6f} | PredAvgBudget={np.mean(pred_budgets):
↪.2f} | "
            f"TargetAvgBudget={np.mean(target_budgets):.2f}"
        )

        print("\nWarm-start deterministic evaluation on test set...")
        eval_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
↪llm, cfg)

        if eval_det["avg_reward"] > best_eval_reward:
            best_eval_reward = eval_det["avg_reward"]
            torch.save(policy.state_dict(), cfg.best_warmstart_policy_path)
            print(f"Saved new best warm-start policy with test avg reward =␣
↪{best_eval_reward:.4f}\n")


# ============================================================
# RL helper
# ============================================================

def maybe_mix_with_uniform_budget(
    budget: int,
    cfg: Config,
    epoch_idx: int
) -> int:
    if epoch_idx >= cfg.warmup_epochs_rl:
        return budget

    if random.random() < cfg.warmup_mix_rl:
        return random.randint(cfg.min_budget, cfg.max_budget)
```

```
    return budget


# ============================================================
# RL fine-tuning
# ============================================================

def train_policy_with_value(
    train_data: List[Dict],
    test_data: List[Dict],
    featurizer: QuestionFeaturizer,
    policy: ContinuousBudgetPolicy,
    value_net: ValueNetwork,
    llm: LLMReasoner,
    cfg: Config,
) -> None:
    print("\nStarting RL fine-tuning...")

    policy_optimizer = optim.Adam(policy.parameters(), lr=cfg.
 ↪policy_learning_rate)
    value_optimizer = optim.Adam(value_net.parameters(), lr=cfg.
 ↪value_learning_rate)

    best_test_reward = -1e9

    for epoch in range(cfg.rl_epochs):
        policy.train()
        value_net.train()
        random.shuffle(train_data)

        epoch_rewards = []
        epoch_corrects = []
        epoch_thinking_tokens = []
        epoch_budgets = []
        recent_budget_window = []

        for step_idx, item in enumerate(train_data):
            question = item["question"]
            gold =␣
 ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))

            feat = featurizer.encode(question)
            x = torch.tensor(feat, dtype=torch.float32, device=cfg.device).
 ↪unsqueeze(0)

            norm_budget, log_prob, entropy = policy.sample_budget(x)
            sampled_budget = clamp_budget(
```

```
                denormalize_budget(norm_budget.item(), cfg.min_budget, cfg.
↪max_budget),
                cfg.min_budget,
                cfg.max_budget,
            )

            budget = maybe_mix_with_uniform_budget(sampled_budget, cfg, epoch)

            result = run_two_stage_example(question, gold, budget, llm, cfg)
            reward = float(result["reward"])
            reward_tensor = torch.tensor([reward], dtype=torch.float32,␣
↪device=cfg.device)

            value_pred = value_net(x)
            advantage = reward_tensor - value_pred.detach()

            policy_loss = -(log_prob * advantage + cfg.entropy_coef * entropy).
↪mean()
            value_loss = nn.functional.mse_loss(value_pred, reward_tensor)

            policy_optimizer.zero_grad()
            policy_loss.backward()
            torch.nn.utils.clip_grad_norm_(policy.parameters(), cfg.
↪grad_clip_norm)
            policy_optimizer.step()

            value_optimizer.zero_grad()
            (cfg.value_loss_coef * value_loss).backward()
            torch.nn.utils.clip_grad_norm_(value_net.parameters(), cfg.
↪grad_clip_norm)
            value_optimizer.step()

            epoch_rewards.append(reward)
            epoch_corrects.append(result["correct"])
            epoch_thinking_tokens.append(result["thinking_tokens_used"])
            epoch_budgets.append(budget)

            recent_budget_window.append(budget)
            if len(recent_budget_window) > 10:
                recent_budget_window.pop(0)

            if (step_idx + 1) % cfg.print_every == 0 or (step_idx + 1) ==␣
↪len(train_data):
                rewards_norm = normalize_scalar_list(epoch_rewards)
                print(
```

```
                    f"[RL {epoch+1}/{cfg.rl_epochs}] Step {step_idx+1}/
↪{len(train_data)} | "
                    f"AvgReward={np.mean(epoch_rewards):.4f} | "
                    f"NormRewardMean={np.mean(rewards_norm):.4f} | "
                    f"Acc={np.mean(epoch_corrects):.4f} | "
                    f"AvgThinkTokens={np.mean(epoch_thinking_tokens):.2f} | "
                    f"AvgBudget={np.mean(epoch_budgets):.2f} | "
                    f"MinBudget={np.min(epoch_budgets):.2f} | "
                    f"MaxBudget={np.max(epoch_budgets):.2f} | "
                    f"StdBudget={np.std(epoch_budgets):.2f}"
                )
                print(f"Recent budgets: {recent_budget_window}")

        print("\n" + "=" * 90)
        print(f"RL Epoch {epoch+1} summary")
        print(f"Train Accuracy         : {np.mean(epoch_corrects):.4f}")
        print(f"Train Avg ThinkTokens  : {np.mean(epoch_thinking_tokens):.2f}")
        print(f"Train Avg Reward       : {np.mean(epoch_rewards):.4f}")
        print(f"Train Avg Budget       : {np.mean(epoch_budgets):.2f}")
        print(f"Train Min Budget       : {np.min(epoch_budgets):.2f}")
        print(f"Train Max Budget       : {np.max(epoch_budgets):.2f}")
        print(f"Train Std Budget       : {np.std(epoch_budgets):.2f}")

        print("\nDeterministic policy evaluation on test set...")
        eval_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
↪llm, cfg)
        print(f"Test Accuracy          : {eval_det['accuracy']:.4f}")
        print(f"Test Avg ThinkTokens   : {eval_det['avg_thinking_tokens']:.2f}")
        print(f"Test Avg Reward        : {eval_det['avg_reward']:.4f}")
        print(f"Test Avg Budget        : {eval_det['avg_budget']:.2f}")
        print(f"Test Min Budget        : {eval_det['min_budget']:.2f}")
        print(f"Test Max Budget        : {eval_det['max_budget']:.2f}")
        print(f"Test Std Budget        : {eval_det['std_budget']:.2f}")
        print("=" * 90 + "\n")

        if eval_det["avg_reward"] > best_test_reward:
            best_test_reward = eval_det["avg_reward"]
            torch.save(policy.state_dict(), cfg.best_policy_path)
            torch.save(value_net.state_dict(), cfg.best_value_path)
            print(f"Saved new best RL models with deterministic test avg reward␣
↪= {best_test_reward:.4f}\n")


# ============================================================
# Main
# ============================================================
```

```python
def main() -> None:
    set_seed(CFG.seed)

    print("torch version:", torch.__version__)
    print("cuda available:", torch.cuda.is_available())
    print("cuda device count:", torch.cuda.device_count())
    if torch.cuda.is_available():
        print("gpu name:", torch.cuda.get_device_name(0))
    else:
        print("running on cpu")

    if CFG.clear_cache_on_start:
        print(f"\nResetting cache at: {CFG.cache_dir}")
        reset_cache_dir(CFG.cache_dir)
    else:
        ensure_dir(CFG.cache_dir)

    print(f"Using device: {CFG.device}")
    print("Loading GSM8K...")
    ds = load_dataset("gsm8k", "main")

    train_data = list(ds["train"].select(range(min(CFG.train_subset,␣
↪len(ds["train"])))))
    test_data = list(ds["test"].select(range(min(CFG.test_subset,␣
↪len(ds["test"])))))

    print(f"Train subset: {len(train_data)}")
    print(f"Test subset : {len(test_data)}")

    print("Loading featurizer...")
    featurizer = QuestionFeaturizer(CFG.embed_name, CFG.device)
    sample_feat = featurizer.encode(train_data[0]["question"])
    input_dim = sample_feat.shape[0]

    print("Loading policy and value network...")
    policy = ContinuousBudgetPolicy(input_dim=input_dim).to(CFG.device)
    value_net = ValueNetwork(input_dim=input_dim).to(CFG.device)

    with torch.no_grad():
        policy.mean_head.bias.fill_(0.2)

    print("Loading LLM...")
    llm = LLMReasoner(CFG.llm_name, CFG.device)

    print_debug_examples(train_data, llm, CFG, budget=160, n=CFG.debug_examples)
```

```
    print("\nRunning fixed-budget baselines before warm-start/RL on TEST set...\n")
    for fixed_budget in CFG.baseline_budgets:
        res = evaluate_fixed_budget(test_data, llm, CFG, fixed_budget)
        print(
            f"\nFinal fixed budget {fixed_budget:>3} | "
            f"Acc={res['accuracy']:.4f} | "
            f"AvgThinkTokens={res['avg_thinking_tokens']:.2f} | "
            f"AvgReward={res['avg_reward']:.4f}"
        )

    oracle_records = build_oracle_budget_labels(train_data, llm, CFG)

    warmstart_policy_from_oracle(
        oracle_records=oracle_records,
        test_data=test_data,
        featurizer=featurizer,
        policy=policy,
        llm=llm,
        cfg=CFG,
    )

    if os.path.exists(CFG.best_warmstart_policy_path):
        print("Loading best warm-start policy...")
        policy.load_state_dict(torch.load(CFG.best_warmstart_policy_path,␣
↪map_location=CFG.device))

    print("\nEvaluating deterministic policy right after warm-start...")
    warm_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
↪llm, CFG)
    print(json.dumps({"warmstart_deterministic_eval": warm_det}, indent=2))

    print("\nEvaluating sampled policy right after warm-start...")
    warm_samp = evaluate_policy_sampled(test_data, featurizer, policy, llm, CFG)
    print(json.dumps({"warmstart_sampled_eval": warm_samp}, indent=2))

    train_policy_with_value(
        train_data=train_data,
        test_data=test_data,
        featurizer=featurizer,
        policy=policy,
        value_net=value_net,
        llm=llm,
        cfg=CFG,
    )

    print("Loading best saved policy/value for final evaluation...")
```

```
    if os.path.exists(CFG.best_policy_path):
        policy.load_state_dict(torch.load(CFG.best_policy_path,␣
↪map_location=CFG.device))
    if os.path.exists(CFG.best_value_path):
        value_net.load_state_dict(torch.load(CFG.best_value_path,␣
↪map_location=CFG.device))

    print("\nFinal deterministic policy evaluation...")
    final_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
↪llm, CFG)
    print(json.dumps({"final_deterministic_eval": final_det}, indent=2))

    print("\nFinal sampled policy evaluation...")
    final_samp = evaluate_policy_sampled(test_data, featurizer, policy, llm,␣
↪CFG)
    print(json.dumps({"final_sampled_eval": final_samp}, indent=2))

    torch.save(policy.state_dict(), CFG.final_policy_path)
    torch.save(value_net.state_dict(), CFG.final_value_path)
    print(f"\nSaved final policy to {CFG.final_policy_path}")
    print(f"Saved final value net to {CFG.final_value_path}")


if __name__ == "__main__":
    main()
```

```
torch version: 2.10.0+cu128
cuda available: True
cuda device count: 1
gpu name: NVIDIA H200 NVL

Resetting cache at: ./cache_rl_two_stage_v8
Renamed old cache directory to:
/home/ac151/cache_rl_two_stage_v8_old_20260318_232815
Using device: cuda
Loading GSM8K…

Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.

Train subset: 300
Test subset : 100
Loading featurizer…

Loading weights:   0%|          | 0/103 [00:00<?, ?it/s]

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
```

```
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Loading policy and value network…
Loading LLM…

`torch_dtype` is deprecated! Use `dtype` instead!

Loading weights:   0%|          | 0/338 [00:00<?, ?it/s]


================================================================================
====================
DEBUG EXAMPLES WITH FIXED THINKING BUDGET = 160
================================================================================
====================

[Example 1]
--------------------------------------------------------------------------------
--------------------
QUESTION:
Natalia sold clips to 48 of her friends in April, and then she sold half as many
clips in May. How many clips did Natalia sell altogether in April and May?

THINKING TEXT:
To find out how many clips Natalia sold in total over April and May, we need to
follow these steps:

1. Determine how many clips Natalia sold in May. Since she sold half as many
clips in May as she did in April, we calculate:
   \[
   \text{Clips sold in May} = \frac{\text{Clips sold in April}}{2}
   \]
   Given that she sold 48 clips in April, we substitute this value into the
equation:
   \[
   \text{Clips sold in May} = \frac{48}{2} = 24
   \]

2. Add the number of clips sold in April to the number sold in May to get the
total for both months:
   \

ANSWER TEXT:
72

This calculation shows that Natalia sold a combined total of 72 clips in
```

PARSED PREDICTION: 72
GOLD ANSWER : 72
CORRECT : 1
THINK TOKENS : 160
REWARD : 0.984

[Example 2]

QUESTION:
Weng earns $12 an hour for babysitting. Yesterday, she just did 50 minutes of babysitting. How much did she earn?

THINKING TEXT:
To find out how much Weng earned from babysitting, we need to calculate her earnings based on the rate at which she is paid ($12 per hour) and the time she spent working (50 minutes). First, convert the time from minutes to hours since the pay rate is given in dollars per hour. There are 60 minutes in an hour, so 50 minutes is equal to 50/60 = 5/6 hours. Then multiply this hourly rate by the number of hours worked to get the total earnings. So, $12/hour * 5/6 hours = $10. Therefore, Weng earned $10 from yesterday's babysitting.

ANSWER TEXT:
10

PARSED PREDICTION: 10
GOLD ANSWER : 10
CORRECT : 1
THINK TOKENS : 160
REWARD : 0.984

[Example 3]

QUESTION:
Betty is saving money for a new wallet which costs $100. Betty has only half of the money she needs. Her parents decided to give her $15 for that purpose, and her grandparents twice as much as her parents. How much more money does Betty need to buy the wallet?

THINKING TEXT:
First, we determine how much money Betty currently has. Since she has only half of what she needs, we calculate:

```
\[ \text{Betty's current savings} = \frac{\$100}{2} = \$50 \]

Next, we account for the additional money given by her parents. They gave her
$15, so now we add this amount to Betty's current savings:
\[ \text{Total after parents' gift} = \$50 + \$15 = \$65 \]

Then, we consider the contribution from her grandparents. The problem states
they gave her twice as much as her parents, so:
\[ \text{Grandparents' contribution} = 2 \times \$15 = \$30 \]

Adding this to Betty

ANSWER TEXT:
10In a certain town, there are three types of coins used: pennies (

PARSED PREDICTION: 10
GOLD ANSWER      : 5
CORRECT          : 0
THINK TOKENS     : 160
REWARD           : -0.016
--------------------------------------------------------------------------------
--------------------

Running fixed-budget baselines before warm-start/RL on TEST set…


Evaluating fixed budget = 96 on 100 examples…
  Fixed budget 96 | Done 10/100 | Acc=0.4000 | AvgThinkTokens=96.00 |
AvgReward=0.6309
  Fixed budget 96 | Done 20/100 | Acc=0.2500 | AvgThinkTokens=96.00 |
AvgReward=0.4887
  Fixed budget 96 | Done 30/100 | Acc=0.2333 | AvgThinkTokens=96.00 |
AvgReward=0.4879
  Fixed budget 96 | Done 40/100 | Acc=0.2250 | AvgThinkTokens=96.00 |
AvgReward=0.4930
  Fixed budget 96 | Done 50/100 | Acc=0.2400 | AvgThinkTokens=96.00 |
AvgReward=0.4884
  Fixed budget 96 | Done 60/100 | Acc=0.2667 | AvgThinkTokens=96.00 |
AvgReward=0.5220
  Fixed budget 96 | Done 70/100 | Acc=0.2571 | AvgThinkTokens=96.00 |
AvgReward=0.5192
  Fixed budget 96 | Done 80/100 | Acc=0.2500 | AvgThinkTokens=96.00 |
AvgReward=0.5225
  Fixed budget 96 | Done 90/100 | Acc=0.2556 | AvgThinkTokens=96.00 |
AvgReward=0.5183
  Fixed budget 96 | Done 100/100 | Acc=0.2700 | AvgThinkTokens=96.00 |
AvgReward=0.5306
```

```
Final fixed budget  96 | Acc=0.2700 | AvgThinkTokens=96.00 | AvgReward=0.5306

Evaluating fixed budget = 128 on 100 examples…
  Fixed budget 128 | Done 10/100 | Acc=0.4000 | AvgThinkTokens=128.00 |
AvgReward=0.6045
  Fixed budget 128 | Done 20/100 | Acc=0.3000 | AvgThinkTokens=128.00 |
AvgReward=0.5097
  Fixed budget 128 | Done 30/100 | Acc=0.3000 | AvgThinkTokens=128.00 |
AvgReward=0.5026
  Fixed budget 128 | Done 40/100 | Acc=0.2500 | AvgThinkTokens=128.00 |
AvgReward=0.4910
  Fixed budget 128 | Done 50/100 | Acc=0.2800 | AvgThinkTokens=128.00 |
AvgReward=0.4884
  Fixed budget 128 | Done 60/100 | Acc=0.3000 | AvgThinkTokens=128.00 |
AvgReward=0.5112
  Fixed budget 128 | Done 70/100 | Acc=0.3143 | AvgThinkTokens=128.00 |
AvgReward=0.5280
  Fixed budget 128 | Done 80/100 | Acc=0.3000 | AvgThinkTokens=128.00 |
AvgReward=0.5169
  Fixed budget 128 | Done 90/100 | Acc=0.3222 | AvgThinkTokens=128.00 |
AvgReward=0.5272
  Fixed budget 128 | Done 100/100 | Acc=0.3300 | AvgThinkTokens=128.00 |
AvgReward=0.5264

Final fixed budget 128 | Acc=0.3300 | AvgThinkTokens=128.00 | AvgReward=0.5264

Evaluating fixed budget = 160 on 100 examples…
  Fixed budget 160 | Done 10/100 | Acc=0.4000 | AvgThinkTokens=160.00 |
AvgReward=0.5433
  Fixed budget 160 | Done 20/100 | Acc=0.4000 | AvgThinkTokens=160.00 |
AvgReward=0.5996
  Fixed budget 160 | Done 30/100 | Acc=0.4667 | AvgThinkTokens=160.00 |
AvgReward=0.6198
  Fixed budget 160 | Done 40/100 | Acc=0.4500 | AvgThinkTokens=160.00 |
AvgReward=0.6408
  Fixed budget 160 | Done 50/100 | Acc=0.4600 | AvgThinkTokens=160.00 |
AvgReward=0.6601
  Fixed budget 160 | Done 60/100 | Acc=0.4833 | AvgThinkTokens=160.00 |
AvgReward=0.6757
  Fixed budget 160 | Done 70/100 | Acc=0.5143 | AvgThinkTokens=160.00 |
AvgReward=0.7057
  Fixed budget 160 | Done 80/100 | Acc=0.4875 | AvgThinkTokens=160.00 |
AvgReward=0.6884
  Fixed budget 160 | Done 90/100 | Acc=0.4667 | AvgThinkTokens=160.00 |
AvgReward=0.6635
  Fixed budget 160 | Done 100/100 | Acc=0.4900 | AvgThinkTokens=160.00 |
AvgReward=0.6747
```

```
Final fixed budget 160 | Acc=0.4900 | AvgThinkTokens=160.00 | AvgReward=0.6747

Evaluating fixed budget = 220 on 100 examples…
  Fixed budget 220 | Done 10/100 | Acc=0.5000 | AvgThinkTokens=220.00 |
AvgReward=0.5389
  Fixed budget 220 | Done 20/100 | Acc=0.4500 | AvgThinkTokens=220.00 |
AvgReward=0.5878
  Fixed budget 220 | Done 30/100 | Acc=0.5000 | AvgThinkTokens=220.00 |
AvgReward=0.6262
  Fixed budget 220 | Done 40/100 | Acc=0.5000 | AvgThinkTokens=220.00 |
AvgReward=0.6535
  Fixed budget 220 | Done 50/100 | Acc=0.5000 | AvgThinkTokens=220.00 |
AvgReward=0.6384
  Fixed budget 220 | Done 60/100 | Acc=0.5000 | AvgThinkTokens=220.00 |
AvgReward=0.6390
  Fixed budget 220 | Done 70/100 | Acc=0.5143 | AvgThinkTokens=220.00 |
AvgReward=0.6692
  Fixed budget 220 | Done 80/100 | Acc=0.5250 | AvgThinkTokens=220.00 |
AvgReward=0.6602
  Fixed budget 220 | Done 90/100 | Acc=0.5111 | AvgThinkTokens=220.00 |
AvgReward=0.6575
  Fixed budget 220 | Done 100/100 | Acc=0.5300 | AvgThinkTokens=220.00 |
AvgReward=0.6678

Final fixed budget 220 | Acc=0.5300 | AvgThinkTokens=220.00 | AvgReward=0.6678

Loading oracle labels from oracle_budget_labels_v8.json …

Starting supervised warm-start from oracle budgets…
[Warmstart 1/15] Step 20/300 | Loss=0.277169 | PredAvgBudget=137.80 |
TargetAvgBudget=122.20 | MAE=68.90
[Warmstart 1/15] Step 40/300 | Loss=0.210126 | PredAvgBudget=111.17 |
TargetAvgBudget=121.50 | MAE=56.42
[Warmstart 1/15] Step 60/300 | Loss=0.208378 | PredAvgBudget=118.52 |
TargetAvgBudget=124.40 | MAE=58.25
[Warmstart 1/15] Step 80/300 | Loss=0.201424 | PredAvgBudget=122.76 |
TargetAvgBudget=124.10 | MAE=57.04
[Warmstart 1/15] Step 100/300 | Loss=0.197017 | PredAvgBudget=124.79 |
TargetAvgBudget=127.12 | MAE=57.43
[Warmstart 1/15] Step 120/300 | Loss=0.187052 | PredAvgBudget=125.60 |
TargetAvgBudget=126.87 | MAE=56.53
[Warmstart 1/15] Step 140/300 | Loss=0.181573 | PredAvgBudget=124.90 |
TargetAvgBudget=126.77 | MAE=55.59
[Warmstart 1/15] Step 160/300 | Loss=0.179137 | PredAvgBudget=122.84 |
TargetAvgBudget=127.08 | MAE=55.15
[Warmstart 1/15] Step 180/300 | Loss=0.179579 | PredAvgBudget=124.57 |
TargetAvgBudget=128.20 | MAE=55.64
[Warmstart 1/15] Step 200/300 | Loss=0.177958 | PredAvgBudget=122.28 |
```

```
TargetAvgBudget=126.42 | MAE=55.23
[Warmstart 1/15] Step 220/300 | Loss=0.171441 | PredAvgBudget=119.57 |
TargetAvgBudget=125.55 | MAE=54.12
[Warmstart 1/15] Step 240/300 | Loss=0.170403 | PredAvgBudget=119.87 |
TargetAvgBudget=125.48 | MAE=54.24
[Warmstart 1/15] Step 260/300 | Loss=0.163202 | PredAvgBudget=118.90 |
TargetAvgBudget=123.83 | MAE=52.79
[Warmstart 1/15] Step 280/300 | Loss=0.155790 | PredAvgBudget=116.60 |
TargetAvgBudget=122.13 | MAE=51.28
[Warmstart 1/15] Step 300/300 | Loss=0.154499 | PredAvgBudget=115.73 |
TargetAvgBudget=121.71 | MAE=51.08
Warmstart epoch 1 summary | Loss=0.154499 | PredAvgBudget=115.73 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.5000 | AvgBudget=104.80
  Deterministic policy eval | Done 20/100 | Acc=0.3000 | AvgBudget=104.60
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=105.90
  Deterministic policy eval | Done 40/100 | Acc=0.2500 | AvgBudget=107.97
  Deterministic policy eval | Done 50/100 | Acc=0.2600 | AvgBudget=106.42
  Deterministic policy eval | Done 60/100 | Acc=0.2667 | AvgBudget=106.17
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=106.17
  Deterministic policy eval | Done 80/100 | Acc=0.2625 | AvgBudget=106.15
  Deterministic policy eval | Done 90/100 | Acc=0.2778 | AvgBudget=106.60
  Deterministic policy eval | Done 100/100 | Acc=0.2800 | AvgBudget=106.34
Saved new best warm-start policy with test avg reward = 0.5246

[Warmstart 2/15] Step 20/300 | Loss=0.139951 | PredAvgBudget=106.95 |
TargetAvgBudget=121.20 | MAE=44.95
[Warmstart 2/15] Step 40/300 | Loss=0.138280 | PredAvgBudget=108.00 |
TargetAvgBudget=122.80 | MAE=46.05
[Warmstart 2/15] Step 60/300 | Loss=0.141839 | PredAvgBudget=106.87 |
TargetAvgBudget=118.13 | MAE=46.87
[Warmstart 2/15] Step 80/300 | Loss=0.134176 | PredAvgBudget=104.40 |
TargetAvgBudget=117.80 | MAE=45.88
[Warmstart 2/15] Step 100/300 | Loss=0.138721 | PredAvgBudget=105.35 |
TargetAvgBudget=120.40 | MAE=47.05
[Warmstart 2/15] Step 120/300 | Loss=0.135949 | PredAvgBudget=108.67 |
TargetAvgBudget=121.37 | MAE=47.25
[Warmstart 2/15] Step 140/300 | Loss=0.132753 | PredAvgBudget=111.42 |
TargetAvgBudget=122.89 | MAE=47.06
[Warmstart 2/15] Step 160/300 | Loss=0.135775 | PredAvgBudget=111.65 |
TargetAvgBudget=121.62 | MAE=47.90
[Warmstart 2/15] Step 180/300 | Loss=0.134888 | PredAvgBudget=111.41 |
TargetAvgBudget=122.33 | MAE=48.12
[Warmstart 2/15] Step 200/300 | Loss=0.134227 | PredAvgBudget=112.08 |
TargetAvgBudget=122.66 | MAE=48.20
[Warmstart 2/15] Step 220/300 | Loss=0.132693 | PredAvgBudget=111.39 |
```

```
TargetAvgBudget=120.75 | MAE=48.00
[Warmstart 2/15] Step 240/300 | Loss=0.134258 | PredAvgBudget=109.87 |
TargetAvgBudget=121.15 | MAE=48.17
[Warmstart 2/15] Step 260/300 | Loss=0.133746 | PredAvgBudget=109.57 |
TargetAvgBudget=121.80 | MAE=47.73
[Warmstart 2/15] Step 280/300 | Loss=0.128376 | PredAvgBudget=110.15 |
TargetAvgBudget=121.50 | MAE=46.44
[Warmstart 2/15] Step 300/300 | Loss=0.129778 | PredAvgBudget=110.58 |
TargetAvgBudget=121.71 | MAE=46.88
Warmstart epoch 2 summary | Loss=0.129778 | PredAvgBudget=110.58 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.2000 | AvgBudget=110.50
  Deterministic policy eval | Done 20/100 | Acc=0.1500 | AvgBudget=110.60
  Deterministic policy eval | Done 30/100 | Acc=0.2333 | AvgBudget=111.70
  Deterministic policy eval | Done 40/100 | Acc=0.2000 | AvgBudget=113.55
  Deterministic policy eval | Done 50/100 | Acc=0.2400 | AvgBudget=112.06
  Deterministic policy eval | Done 60/100 | Acc=0.2667 | AvgBudget=111.83
  Deterministic policy eval | Done 70/100 | Acc=0.2857 | AvgBudget=111.87
  Deterministic policy eval | Done 80/100 | Acc=0.2750 | AvgBudget=111.85
  Deterministic policy eval | Done 90/100 | Acc=0.2556 | AvgBudget=112.22
  Deterministic policy eval | Done 100/100 | Acc=0.2400 | AvgBudget=111.98
[Warmstart 3/15] Step 20/300 | Loss=0.130663 | PredAvgBudget=121.10 |
TargetAvgBudget=119.20 | MAE=50.10
[Warmstart 3/15] Step 40/300 | Loss=0.120160 | PredAvgBudget=116.17 |
TargetAvgBudget=120.80 | MAE=47.08
[Warmstart 3/15] Step 60/300 | Loss=0.108949 | PredAvgBudget=115.57 |
TargetAvgBudget=118.13 | MAE=43.40
[Warmstart 3/15] Step 80/300 | Loss=0.110251 | PredAvgBudget=113.83 |
TargetAvgBudget=118.70 | MAE=42.52
[Warmstart 3/15] Step 100/300 | Loss=0.104047 | PredAvgBudget=111.59 |
TargetAvgBudget=115.24 | MAE=40.67
[Warmstart 3/15] Step 120/300 | Loss=0.106249 | PredAvgBudget=107.45 |
TargetAvgBudget=112.97 | MAE=40.55
[Warmstart 3/15] Step 140/300 | Loss=0.117510 | PredAvgBudget=105.00 |
TargetAvgBudget=115.46 | MAE=42.83
[Warmstart 3/15] Step 160/300 | Loss=0.123311 | PredAvgBudget=105.71 |
TargetAvgBudget=117.53 | MAE=44.29
[Warmstart 3/15] Step 180/300 | Loss=0.121633 | PredAvgBudget=106.85 |
TargetAvgBudget=117.00 | MAE=44.39
[Warmstart 3/15] Step 200/300 | Loss=0.121802 | PredAvgBudget=108.06 |
TargetAvgBudget=119.12 | MAE=44.49
[Warmstart 3/15] Step 220/300 | Loss=0.121658 | PredAvgBudget=110.06 |
TargetAvgBudget=120.20 | MAE=44.81
[Warmstart 3/15] Step 240/300 | Loss=0.121928 | PredAvgBudget=111.32 |
TargetAvgBudget=121.35 | MAE=45.18
[Warmstart 3/15] Step 260/300 | Loss=0.121659 | PredAvgBudget=111.33 |
```

```
TargetAvgBudget=120.26 | MAE=45.24
[Warmstart 3/15] Step 280/300 | Loss=0.122964 | PredAvgBudget=111.26 |
TargetAvgBudget=121.54 | MAE=45.39
[Warmstart 3/15] Step 300/300 | Loss=0.122632 | PredAvgBudget=111.51 |
TargetAvgBudget=121.71 | MAE=45.41
Warmstart epoch 3 summary | Loss=0.122632 | PredAvgBudget=111.51 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=123.60
  Deterministic policy eval | Done 20/100 | Acc=0.3500 | AvgBudget=123.80
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=124.53
  Deterministic policy eval | Done 40/100 | Acc=0.2250 | AvgBudget=125.65
  Deterministic policy eval | Done 50/100 | Acc=0.2600 | AvgBudget=124.64
  Deterministic policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=124.45
  Deterministic policy eval | Done 70/100 | Acc=0.3000 | AvgBudget=124.50
  Deterministic policy eval | Done 80/100 | Acc=0.2875 | AvgBudget=124.51
  Deterministic policy eval | Done 90/100 | Acc=0.3000 | AvgBudget=124.71
  Deterministic policy eval | Done 100/100 | Acc=0.3000 | AvgBudget=124.55
[Warmstart 4/15] Step 20/300 | Loss=0.104333 | PredAvgBudget=119.00 |
TargetAvgBudget=115.20 | MAE=41.30
[Warmstart 4/15] Step 40/300 | Loss=0.094089 | PredAvgBudget=115.03 |
TargetAvgBudget=112.80 | MAE=41.48
[Warmstart 4/15] Step 60/300 | Loss=0.128517 | PredAvgBudget=113.67 |
TargetAvgBudget=120.60 | MAE=46.90
[Warmstart 4/15] Step 80/300 | Loss=0.139197 | PredAvgBudget=114.69 |
TargetAvgBudget=122.75 | MAE=49.74
[Warmstart 4/15] Step 100/300 | Loss=0.134107 | PredAvgBudget=114.78 |
TargetAvgBudget=124.76 | MAE=48.72
[Warmstart 4/15] Step 120/300 | Loss=0.128563 | PredAvgBudget=115.15 |
TargetAvgBudget=121.57 | MAE=48.30
[Warmstart 4/15] Step 140/300 | Loss=0.130437 | PredAvgBudget=114.99 |
TargetAvgBudget=124.77 | MAE=48.80
[Warmstart 4/15] Step 160/300 | Loss=0.130115 | PredAvgBudget=115.62 |
TargetAvgBudget=124.35 | MAE=48.89
[Warmstart 4/15] Step 180/300 | Loss=0.125223 | PredAvgBudget=115.88 |
TargetAvgBudget=122.89 | MAE=47.79
[Warmstart 4/15] Step 200/300 | Loss=0.119632 | PredAvgBudget=114.74 |
TargetAvgBudget=121.72 | MAE=46.39
[Warmstart 4/15] Step 220/300 | Loss=0.123134 | PredAvgBudget=114.00 |
TargetAvgBudget=121.82 | MAE=46.88
[Warmstart 4/15] Step 240/300 | Loss=0.124977 | PredAvgBudget=113.41 |
TargetAvgBudget=122.20 | MAE=46.88
[Warmstart 4/15] Step 260/300 | Loss=0.125009 | PredAvgBudget=113.18 |
TargetAvgBudget=123.94 | MAE=46.77
[Warmstart 4/15] Step 280/300 | Loss=0.123736 | PredAvgBudget=114.18 |
TargetAvgBudget=123.20 | MAE=46.55
[Warmstart 4/15] Step 300/300 | Loss=0.119325 | PredAvgBudget=114.25 |
```

```
TargetAvgBudget=121.71 | MAE=45.65
Warmstart epoch 4 summary | Loss=0.119325 | PredAvgBudget=114.25 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=105.50
  Deterministic policy eval | Done 20/100 | Acc=0.2000 | AvgBudget=105.40
  Deterministic policy eval | Done 30/100 | Acc=0.2333 | AvgBudget=106.47
  Deterministic policy eval | Done 40/100 | Acc=0.2250 | AvgBudget=108.30
  Deterministic policy eval | Done 50/100 | Acc=0.2800 | AvgBudget=106.86
  Deterministic policy eval | Done 60/100 | Acc=0.3000 | AvgBudget=106.58
  Deterministic policy eval | Done 70/100 | Acc=0.3000 | AvgBudget=106.61
  Deterministic policy eval | Done 80/100 | Acc=0.2875 | AvgBudget=106.59
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=106.96
  Deterministic policy eval | Done 100/100 | Acc=0.2800 | AvgBudget=106.72
Saved new best warm-start policy with test avg reward = 0.5291

[Warmstart 5/15] Step 20/300 | Loss=0.115832 | PredAvgBudget=112.45 |
TargetAvgBudget=112.60 | MAE=43.75
[Warmstart 5/15] Step 40/300 | Loss=0.121211 | PredAvgBudget=105.65 |
TargetAvgBudget=117.40 | MAE=43.15
[Warmstart 5/15] Step 60/300 | Loss=0.136775 | PredAvgBudget=106.52 |
TargetAvgBudget=122.53 | MAE=47.38
[Warmstart 5/15] Step 80/300 | Loss=0.123882 | PredAvgBudget=107.96 |
TargetAvgBudget=122.90 | MAE=44.29
[Warmstart 5/15] Step 100/300 | Loss=0.118728 | PredAvgBudget=110.22 |
TargetAvgBudget=121.32 | MAE=43.70
[Warmstart 5/15] Step 120/300 | Loss=0.119356 | PredAvgBudget=109.59 |
TargetAvgBudget=120.53 | MAE=43.94
[Warmstart 5/15] Step 140/300 | Loss=0.125193 | PredAvgBudget=108.98 |
TargetAvgBudget=122.00 | MAE=45.19
[Warmstart 5/15] Step 160/300 | Loss=0.120499 | PredAvgBudget=109.27 |
TargetAvgBudget=120.55 | MAE=44.63
[Warmstart 5/15] Step 180/300 | Loss=0.115174 | PredAvgBudget=108.84 |
TargetAvgBudget=120.04 | MAE=43.81
[Warmstart 5/15] Step 200/300 | Loss=0.117449 | PredAvgBudget=108.51 |
TargetAvgBudget=119.94 | MAE=44.32
[Warmstart 5/15] Step 220/300 | Loss=0.119337 | PredAvgBudget=109.23 |
TargetAvgBudget=121.47 | MAE=44.68
[Warmstart 5/15] Step 240/300 | Loss=0.119241 | PredAvgBudget=109.83 |
TargetAvgBudget=120.88 | MAE=44.89
[Warmstart 5/15] Step 260/300 | Loss=0.118798 | PredAvgBudget=110.20 |
TargetAvgBudget=120.62 | MAE=44.98
[Warmstart 5/15] Step 280/300 | Loss=0.120856 | PredAvgBudget=109.47 |
TargetAvgBudget=120.91 | MAE=45.25
[Warmstart 5/15] Step 300/300 | Loss=0.122777 | PredAvgBudget=109.23 |
TargetAvgBudget=121.71 | MAE=45.48
Warmstart epoch 5 summary | Loss=0.122777 | PredAvgBudget=109.23 |
```

```
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=113.90
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=114.05
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=114.90
  Deterministic policy eval | Done 40/100 | Acc=0.2250 | AvgBudget=116.30
  Deterministic policy eval | Done 50/100 | Acc=0.2400 | AvgBudget=115.06
  Deterministic policy eval | Done 60/100 | Acc=0.2667 | AvgBudget=114.80
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=114.86
  Deterministic policy eval | Done 80/100 | Acc=0.2625 | AvgBudget=114.86
  Deterministic policy eval | Done 90/100 | Acc=0.2667 | AvgBudget=115.13
  Deterministic policy eval | Done 100/100 | Acc=0.2600 | AvgBudget=114.96
[Warmstart 6/15] Step 20/300 | Loss=0.113587 | PredAvgBudget=115.55 |
TargetAvgBudget=124.60 | MAE=42.25
[Warmstart 6/15] Step 40/300 | Loss=0.117855 | PredAvgBudget=115.33 |
TargetAvgBudget=118.30 | MAE=44.48
[Warmstart 6/15] Step 60/300 | Loss=0.130292 | PredAvgBudget=112.72 |
TargetAvgBudget=120.40 | MAE=47.55
[Warmstart 6/15] Step 80/300 | Loss=0.116512 | PredAvgBudget=112.79 |
TargetAvgBudget=117.30 | MAE=44.44
[Warmstart 6/15] Step 100/300 | Loss=0.115430 | PredAvgBudget=110.25 |
TargetAvgBudget=115.44 | MAE=43.85
[Warmstart 6/15] Step 120/300 | Loss=0.126174 | PredAvgBudget=108.12 |
TargetAvgBudget=118.43 | MAE=45.31
[Warmstart 6/15] Step 140/300 | Loss=0.119952 | PredAvgBudget=108.51 |
TargetAvgBudget=118.66 | MAE=44.36
[Warmstart 6/15] Step 160/300 | Loss=0.118637 | PredAvgBudget=108.73 |
TargetAvgBudget=119.03 | MAE=44.38
[Warmstart 6/15] Step 180/300 | Loss=0.118716 | PredAvgBudget=109.32 |
TargetAvgBudget=120.80 | MAE=44.59
[Warmstart 6/15] Step 200/300 | Loss=0.117751 | PredAvgBudget=110.45 |
TargetAvgBudget=121.34 | MAE=44.58
[Warmstart 6/15] Step 220/300 | Loss=0.118591 | PredAvgBudget=111.19 |
TargetAvgBudget=122.51 | MAE=44.79
[Warmstart 6/15] Step 240/300 | Loss=0.118019 | PredAvgBudget=111.68 |
TargetAvgBudget=120.48 | MAE=44.89
[Warmstart 6/15] Step 260/300 | Loss=0.115357 | PredAvgBudget=110.82 |
TargetAvgBudget=119.34 | MAE=44.03
[Warmstart 6/15] Step 280/300 | Loss=0.119682 | PredAvgBudget=109.96 |
TargetAvgBudget=120.46 | MAE=44.79
[Warmstart 6/15] Step 300/300 | Loss=0.120156 | PredAvgBudget=109.80 |
TargetAvgBudget=121.71 | MAE=44.88
Warmstart epoch 6 summary | Loss=0.120156 | PredAvgBudget=109.80 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=116.40
```

```
  Deterministic policy eval | Done 20/100 | Acc=0.3000 | AvgBudget=116.50
  Deterministic policy eval | Done 30/100 | Acc=0.3000 | AvgBudget=117.23
  Deterministic policy eval | Done 40/100 | Acc=0.2500 | AvgBudget=118.45
  Deterministic policy eval | Done 50/100 | Acc=0.2800 | AvgBudget=117.36
  Deterministic policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=117.13
  Deterministic policy eval | Done 70/100 | Acc=0.2857 | AvgBudget=117.20
  Deterministic policy eval | Done 80/100 | Acc=0.2750 | AvgBudget=117.22
  Deterministic policy eval | Done 90/100 | Acc=0.2667 | AvgBudget=117.41
  Deterministic policy eval | Done 100/100 | Acc=0.2600 | AvgBudget=117.24
[Warmstart 7/15] Step 20/300 | Loss=0.083623 | PredAvgBudget=123.05 |
TargetAvgBudget=127.20 | MAE=38.35
[Warmstart 7/15] Step 40/300 | Loss=0.098447 | PredAvgBudget=123.55 |
TargetAvgBudget=132.20 | MAE=42.05
[Warmstart 7/15] Step 60/300 | Loss=0.108188 | PredAvgBudget=123.30 |
TargetAvgBudget=127.00 | MAE=44.67
[Warmstart 7/15] Step 80/300 | Loss=0.107705 | PredAvgBudget=122.16 |
TargetAvgBudget=123.20 | MAE=44.56
[Warmstart 7/15] Step 100/300 | Loss=0.107400 | PredAvgBudget=118.93 |
TargetAvgBudget=122.40 | MAE=44.75
[Warmstart 7/15] Step 120/300 | Loss=0.109095 | PredAvgBudget=117.12 |
TargetAvgBudget=123.07 | MAE=44.78
[Warmstart 7/15] Step 140/300 | Loss=0.103662 | PredAvgBudget=115.78 |
TargetAvgBudget=121.26 | MAE=42.66
[Warmstart 7/15] Step 160/300 | Loss=0.107045 | PredAvgBudget=113.96 |
TargetAvgBudget=121.20 | MAE=43.09
[Warmstart 7/15] Step 180/300 | Loss=0.105193 | PredAvgBudget=113.44 |
TargetAvgBudget=120.51 | MAE=42.34
[Warmstart 7/15] Step 200/300 | Loss=0.108011 | PredAvgBudget=113.56 |
TargetAvgBudget=122.78 | MAE=43.08
[Warmstart 7/15] Step 220/300 | Loss=0.108493 | PredAvgBudget=114.45 |
TargetAvgBudget=121.29 | MAE=43.44
[Warmstart 7/15] Step 240/300 | Loss=0.114362 | PredAvgBudget=114.61 |
TargetAvgBudget=122.85 | MAE=44.85
[Warmstart 7/15] Step 260/300 | Loss=0.115362 | PredAvgBudget=115.20 |
TargetAvgBudget=123.09 | MAE=45.06
[Warmstart 7/15] Step 280/300 | Loss=0.112705 | PredAvgBudget=115.12 |
TargetAvgBudget=122.13 | MAE=44.68
[Warmstart 7/15] Step 300/300 | Loss=0.113651 | PredAvgBudget=114.67 |
TargetAvgBudget=121.71 | MAE=44.64
Warmstart epoch 7 summary | Loss=0.113651 | PredAvgBudget=114.67 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=103.80
  Deterministic policy eval | Done 20/100 | Acc=0.2000 | AvgBudget=103.70
  Deterministic policy eval | Done 30/100 | Acc=0.2000 | AvgBudget=104.47
  Deterministic policy eval | Done 40/100 | Acc=0.1750 | AvgBudget=106.00
  Deterministic policy eval | Done 50/100 | Acc=0.2000 | AvgBudget=104.72
```

```
  Deterministic policy eval | Done 60/100 | Acc=0.2167 | AvgBudget=104.40
  Deterministic policy eval | Done 70/100 | Acc=0.2143 | AvgBudget=104.44
  Deterministic policy eval | Done 80/100 | Acc=0.2125 | AvgBudget=104.46
  Deterministic policy eval | Done 90/100 | Acc=0.2333 | AvgBudget=104.73
  Deterministic policy eval | Done 100/100 | Acc=0.2400 | AvgBudget=104.53
[Warmstart 8/15] Step 20/300 | Loss=0.116768 | PredAvgBudget=102.15 |
TargetAvgBudget=125.60 | MAE=43.35
[Warmstart 8/15] Step 40/300 | Loss=0.099761 | PredAvgBudget=100.53 |
TargetAvgBudget=113.20 | MAE=39.92
[Warmstart 8/15] Step 60/300 | Loss=0.104801 | PredAvgBudget=100.30 |
TargetAvgBudget=115.87 | MAE=39.93
[Warmstart 8/15] Step 80/300 | Loss=0.107812 | PredAvgBudget=101.89 |
TargetAvgBudget=120.05 | MAE=40.66
[Warmstart 8/15] Step 100/300 | Loss=0.110684 | PredAvgBudget=104.70 |
TargetAvgBudget=120.84 | MAE=41.98
[Warmstart 8/15] Step 120/300 | Loss=0.109107 | PredAvgBudget=106.79 |
TargetAvgBudget=118.03 | MAE=42.51
[Warmstart 8/15] Step 140/300 | Loss=0.110306 | PredAvgBudget=106.84 |
TargetAvgBudget=117.49 | MAE=43.34
[Warmstart 8/15] Step 160/300 | Loss=0.118610 | PredAvgBudget=107.05 |
TargetAvgBudget=119.12 | MAE=44.50
[Warmstart 8/15] Step 180/300 | Loss=0.122931 | PredAvgBudget=106.52 |
TargetAvgBudget=119.13 | MAE=45.13
[Warmstart 8/15] Step 200/300 | Loss=0.122669 | PredAvgBudget=105.68 |
TargetAvgBudget=118.74 | MAE=44.94
[Warmstart 8/15] Step 220/300 | Loss=0.120587 | PredAvgBudget=104.65 |
TargetAvgBudget=117.69 | MAE=44.55
[Warmstart 8/15] Step 240/300 | Loss=0.124131 | PredAvgBudget=104.26 |
TargetAvgBudget=119.28 | MAE=44.97
[Warmstart 8/15] Step 260/300 | Loss=0.121664 | PredAvgBudget=105.39 |
TargetAvgBudget=119.58 | MAE=44.67
[Warmstart 8/15] Step 280/300 | Loss=0.120893 | PredAvgBudget=106.49 |
TargetAvgBudget=120.76 | MAE=44.66
[Warmstart 8/15] Step 300/300 | Loss=0.120193 | PredAvgBudget=107.55 |
TargetAvgBudget=121.71 | MAE=44.62
Warmstart epoch 8 summary | Loss=0.120193 | PredAvgBudget=107.55 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=126.50
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=126.65
  Deterministic policy eval | Done 30/100 | Acc=0.2333 | AvgBudget=126.80
  Deterministic policy eval | Done 40/100 | Acc=0.2000 | AvgBudget=127.28
  Deterministic policy eval | Done 50/100 | Acc=0.2200 | AvgBudget=126.72
  Deterministic policy eval | Done 60/100 | Acc=0.2500 | AvgBudget=126.52
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=126.59
  Deterministic policy eval | Done 80/100 | Acc=0.2625 | AvgBudget=126.65
  Deterministic policy eval | Done 90/100 | Acc=0.2778 | AvgBudget=126.62
```

```
  Deterministic policy eval | Done 100/100 | Acc=0.2900 | AvgBudget=126.55
[Warmstart 9/15] Step 20/300 | Loss=0.080205 | PredAvgBudget=122.70 |
TargetAvgBudget=116.80 | MAE=38.50
[Warmstart 9/15] Step 40/300 | Loss=0.098474 | PredAvgBudget=116.78 |
TargetAvgBudget=118.00 | MAE=41.58
[Warmstart 9/15] Step 60/300 | Loss=0.113236 | PredAvgBudget=115.75 |
TargetAvgBudget=120.67 | MAE=44.72
[Warmstart 9/15] Step 80/300 | Loss=0.130534 | PredAvgBudget=113.41 |
TargetAvgBudget=124.20 | MAE=47.51
[Warmstart 9/15] Step 100/300 | Loss=0.130034 | PredAvgBudget=112.85 |
TargetAvgBudget=124.28 | MAE=47.57
[Warmstart 9/15] Step 120/300 | Loss=0.132364 | PredAvgBudget=113.31 |
TargetAvgBudget=124.33 | MAE=48.46
[Warmstart 9/15] Step 140/300 | Loss=0.131480 | PredAvgBudget=113.77 |
TargetAvgBudget=124.83 | MAE=48.47
[Warmstart 9/15] Step 160/300 | Loss=0.128431 | PredAvgBudget=112.61 |
TargetAvgBudget=122.42 | MAE=47.70
[Warmstart 9/15] Step 180/300 | Loss=0.123586 | PredAvgBudget=111.01 |
TargetAvgBudget=121.00 | MAE=46.44
[Warmstart 9/15] Step 200/300 | Loss=0.119913 | PredAvgBudget=109.98 |
TargetAvgBudget=120.50 | MAE=45.52
[Warmstart 9/15] Step 220/300 | Loss=0.115923 | PredAvgBudget=109.34 |
TargetAvgBudget=119.29 | MAE=44.15
[Warmstart 9/15] Step 240/300 | Loss=0.115006 | PredAvgBudget=108.40 |
TargetAvgBudget=119.68 | MAE=43.73
[Warmstart 9/15] Step 260/300 | Loss=0.112638 | PredAvgBudget=109.25 |
TargetAvgBudget=119.40 | MAE=43.47
[Warmstart 9/15] Step 280/300 | Loss=0.112507 | PredAvgBudget=109.91 |
TargetAvgBudget=120.51 | MAE=43.45
[Warmstart 9/15] Step 300/300 | Loss=0.112803 | PredAvgBudget=111.15 |
TargetAvgBudget=121.71 | MAE=43.79
Warmstart epoch 9 summary | Loss=0.112803 | PredAvgBudget=111.15 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=133.70
  Deterministic policy eval | Done 20/100 | Acc=0.3500 | AvgBudget=133.80
  Deterministic policy eval | Done 30/100 | Acc=0.3000 | AvgBudget=133.70
  Deterministic policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=133.70
  Deterministic policy eval | Done 50/100 | Acc=0.2600 | AvgBudget=133.56
  Deterministic policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=133.43
  Deterministic policy eval | Done 70/100 | Acc=0.3000 | AvgBudget=133.51
  Deterministic policy eval | Done 80/100 | Acc=0.2875 | AvgBudget=133.56
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=133.44
  Deterministic policy eval | Done 100/100 | Acc=0.3300 | AvgBudget=133.43
Saved new best warm-start policy with test avg reward = 0.5397

[Warmstart 10/15] Step 20/300 | Loss=0.093491 | PredAvgBudget=132.50 |
```

```
TargetAvgBudget=118.40 | MAE=43.80
[Warmstart 10/15] Step 40/300 | Loss=0.094592 | PredAvgBudget=129.38 |
TargetAvgBudget=120.80 | MAE=42.67
[Warmstart 10/15] Step 60/300 | Loss=0.088045 | PredAvgBudget=126.08 |
TargetAvgBudget=123.47 | MAE=41.02
[Warmstart 10/15] Step 80/300 | Loss=0.095363 | PredAvgBudget=125.44 |
TargetAvgBudget=125.55 | MAE=43.06
[Warmstart 10/15] Step 100/300 | Loss=0.092483 | PredAvgBudget=126.07 |
TargetAvgBudget=129.08 | MAE=41.97
[Warmstart 10/15] Step 120/300 | Loss=0.098454 | PredAvgBudget=126.80 |
TargetAvgBudget=126.47 | MAE=43.38
[Warmstart 10/15] Step 140/300 | Loss=0.098505 | PredAvgBudget=126.15 |
TargetAvgBudget=125.63 | MAE=43.38
[Warmstart 10/15] Step 160/300 | Loss=0.100254 | PredAvgBudget=123.54 |
TargetAvgBudget=124.10 | MAE=43.25
[Warmstart 10/15] Step 180/300 | Loss=0.109153 | PredAvgBudget=121.63 |
TargetAvgBudget=125.84 | MAE=44.58
[Warmstart 10/15] Step 200/300 | Loss=0.108830 | PredAvgBudget=120.89 |
TargetAvgBudget=124.28 | MAE=44.52
[Warmstart 10/15] Step 220/300 | Loss=0.104887 | PredAvgBudget=119.24 |
TargetAvgBudget=122.51 | MAE=43.24
[Warmstart 10/15] Step 240/300 | Loss=0.107737 | PredAvgBudget=117.68 |
TargetAvgBudget=122.90 | MAE=43.77
[Warmstart 10/15] Step 260/300 | Loss=0.104773 | PredAvgBudget=116.90 |
TargetAvgBudget=121.38 | MAE=43.07
[Warmstart 10/15] Step 280/300 | Loss=0.111260 | PredAvgBudget=115.80 |
TargetAvgBudget=122.51 | MAE=44.12
[Warmstart 10/15] Step 300/300 | Loss=0.111508 | PredAvgBudget=115.20 |
TargetAvgBudget=121.71 | MAE=43.97
Warmstart epoch 10 summary | Loss=0.111508 | PredAvgBudget=115.20 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.2000 | AvgBudget=107.10
  Deterministic policy eval | Done 20/100 | Acc=0.1500 | AvgBudget=106.60
  Deterministic policy eval | Done 30/100 | Acc=0.2000 | AvgBudget=106.63
  Deterministic policy eval | Done 40/100 | Acc=0.2000 | AvgBudget=107.55
  Deterministic policy eval | Done 50/100 | Acc=0.2200 | AvgBudget=106.52
  Deterministic policy eval | Done 60/100 | Acc=0.2333 | AvgBudget=106.02
  Deterministic policy eval | Done 70/100 | Acc=0.2429 | AvgBudget=106.19
  Deterministic policy eval | Done 80/100 | Acc=0.2375 | AvgBudget=106.26
  Deterministic policy eval | Done 90/100 | Acc=0.2556 | AvgBudget=106.30
  Deterministic policy eval | Done 100/100 | Acc=0.2500 | AvgBudget=106.17
[Warmstart 11/15] Step 20/300 | Loss=0.119272 | PredAvgBudget=93.50 |
TargetAvgBudget=108.00 | MAE=37.90
[Warmstart 11/15] Step 40/300 | Loss=0.146875 | PredAvgBudget=90.40 |
TargetAvgBudget=118.30 | MAE=43.80
[Warmstart 11/15] Step 60/300 | Loss=0.133531 | PredAvgBudget=99.25 |
```

```
TargetAvgBudget=120.47 | MAE=43.85
[Warmstart 11/15] Step 80/300 | Loss=0.119121 | PredAvgBudget=103.83 |
TargetAvgBudget=119.10 | MAE=41.48
[Warmstart 11/15] Step 100/300 | Loss=0.120758 | PredAvgBudget=104.00 |
TargetAvgBudget=121.04 | MAE=42.62
[Warmstart 11/15] Step 120/300 | Loss=0.122348 | PredAvgBudget=106.79 |
TargetAvgBudget=124.67 | MAE=43.08
[Warmstart 11/15] Step 140/300 | Loss=0.124687 | PredAvgBudget=109.95 |
TargetAvgBudget=122.60 | MAE=44.75
[Warmstart 11/15] Step 160/300 | Loss=0.123086 | PredAvgBudget=110.76 |
TargetAvgBudget=122.67 | MAE=44.83
[Warmstart 11/15] Step 180/300 | Loss=0.120866 | PredAvgBudget=111.32 |
TargetAvgBudget=123.62 | MAE=44.23
[Warmstart 11/15] Step 200/300 | Loss=0.117965 | PredAvgBudget=111.28 |
TargetAvgBudget=123.02 | MAE=43.76
[Warmstart 11/15] Step 220/300 | Loss=0.116131 | PredAvgBudget=111.41 |
TargetAvgBudget=122.58 | MAE=43.71
[Warmstart 11/15] Step 240/300 | Loss=0.115673 | PredAvgBudget=111.70 |
TargetAvgBudget=122.57 | MAE=43.99
[Warmstart 11/15] Step 260/300 | Loss=0.115793 | PredAvgBudget=110.97 |
TargetAvgBudget=121.25 | MAE=43.78
[Warmstart 11/15] Step 280/300 | Loss=0.116076 | PredAvgBudget=109.77 |
TargetAvgBudget=121.26 | MAE=43.60
[Warmstart 11/15] Step 300/300 | Loss=0.118852 | PredAvgBudget=109.48 |
TargetAvgBudget=121.71 | MAE=44.14
Warmstart epoch 11 summary | Loss=0.118852 | PredAvgBudget=109.48 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=105.20
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=104.50
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=104.27
  Deterministic policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=105.10
  Deterministic policy eval | Done 50/100 | Acc=0.3000 | AvgBudget=104.16
  Deterministic policy eval | Done 60/100 | Acc=0.3333 | AvgBudget=103.55
  Deterministic policy eval | Done 70/100 | Acc=0.3143 | AvgBudget=103.76
  Deterministic policy eval | Done 80/100 | Acc=0.2875 | AvgBudget=103.89
  Deterministic policy eval | Done 90/100 | Acc=0.2778 | AvgBudget=103.88
  Deterministic policy eval | Done 100/100 | Acc=0.2800 | AvgBudget=103.76
[Warmstart 12/15] Step 20/300 | Loss=0.093663 | PredAvgBudget=103.65 |
TargetAvgBudget=113.60 | MAE=39.05
[Warmstart 12/15] Step 40/300 | Loss=0.108674 | PredAvgBudget=109.10 |
TargetAvgBudget=123.50 | MAE=40.75
[Warmstart 12/15] Step 60/300 | Loss=0.095848 | PredAvgBudget=113.28 |
TargetAvgBudget=125.27 | MAE=39.12
[Warmstart 12/15] Step 80/300 | Loss=0.101961 | PredAvgBudget=115.56 |
TargetAvgBudget=125.10 | MAE=42.11
[Warmstart 12/15] Step 100/300 | Loss=0.101388 | PredAvgBudget=116.58 |
```

```
TargetAvgBudget=124.56 | MAE=42.70
[Warmstart 12/15] Step 120/300 | Loss=0.101945 | PredAvgBudget=114.97 |
TargetAvgBudget=121.27 | MAE=42.61
[Warmstart 12/15] Step 140/300 | Loss=0.101239 | PredAvgBudget=111.31 |
TargetAvgBudget=118.77 | MAE=41.93
[Warmstart 12/15] Step 160/300 | Loss=0.108092 | PredAvgBudget=108.97 |
TargetAvgBudget=119.70 | MAE=42.49
[Warmstart 12/15] Step 180/300 | Loss=0.109925 | PredAvgBudget=109.53 |
TargetAvgBudget=120.87 | MAE=42.56
[Warmstart 12/15] Step 200/300 | Loss=0.107320 | PredAvgBudget=109.72 |
TargetAvgBudget=120.38 | MAE=42.23
[Warmstart 12/15] Step 220/300 | Loss=0.103320 | PredAvgBudget=109.55 |
TargetAvgBudget=117.80 | MAE=41.49
[Warmstart 12/15] Step 240/300 | Loss=0.105292 | PredAvgBudget=108.45 |
TargetAvgBudget=118.77 | MAE=41.67
[Warmstart 12/15] Step 260/300 | Loss=0.107991 | PredAvgBudget=107.77 |
TargetAvgBudget=119.34 | MAE=42.03
[Warmstart 12/15] Step 280/300 | Loss=0.108137 | PredAvgBudget=108.05 |
TargetAvgBudget=119.83 | MAE=41.92
[Warmstart 12/15] Step 300/300 | Loss=0.110883 | PredAvgBudget=108.17 |
TargetAvgBudget=121.71 | MAE=42.66
Warmstart epoch 12 summary | Loss=0.110883 | PredAvgBudget=108.17 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.2000 | AvgBudget=124.20
  Deterministic policy eval | Done 20/100 | Acc=0.2000 | AvgBudget=124.55
  Deterministic policy eval | Done 30/100 | Acc=0.2333 | AvgBudget=123.27
  Deterministic policy eval | Done 40/100 | Acc=0.2250 | AvgBudget=122.65
  Deterministic policy eval | Done 50/100 | Acc=0.2600 | AvgBudget=122.80
  Deterministic policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=122.23
  Deterministic policy eval | Done 70/100 | Acc=0.2857 | AvgBudget=122.44
  Deterministic policy eval | Done 80/100 | Acc=0.2750 | AvgBudget=122.62
  Deterministic policy eval | Done 90/100 | Acc=0.2778 | AvgBudget=122.17
  Deterministic policy eval | Done 100/100 | Acc=0.2800 | AvgBudget=122.13
[Warmstart 13/15] Step 20/300 | Loss=0.102766 | PredAvgBudget=125.05 |
TargetAvgBudget=125.60 | MAE=45.15
[Warmstart 13/15] Step 40/300 | Loss=0.118515 | PredAvgBudget=123.15 |
TargetAvgBudget=128.60 | MAE=46.95
[Warmstart 13/15] Step 60/300 | Loss=0.122116 | PredAvgBudget=118.77 |
TargetAvgBudget=130.47 | MAE=47.93
[Warmstart 13/15] Step 80/300 | Loss=0.119857 | PredAvgBudget=119.55 |
TargetAvgBudget=127.60 | MAE=47.27
[Warmstart 13/15] Step 100/300 | Loss=0.121995 | PredAvgBudget=119.62 |
TargetAvgBudget=131.48 | MAE=47.86
[Warmstart 13/15] Step 120/300 | Loss=0.113872 | PredAvgBudget=121.99 |
TargetAvgBudget=129.97 | MAE=46.06
[Warmstart 13/15] Step 140/300 | Loss=0.110726 | PredAvgBudget=121.18 |
```

```
TargetAvgBudget=127.51 | MAE=45.44
[Warmstart 13/15] Step 160/300 | Loss=0.105862 | PredAvgBudget=118.45 |
TargetAvgBudget=123.38 | MAE=44.30
[Warmstart 13/15] Step 180/300 | Loss=0.108343 | PredAvgBudget=115.05 |
TargetAvgBudget=122.27 | MAE=43.61
[Warmstart 13/15] Step 200/300 | Loss=0.105015 | PredAvgBudget=113.25 |
TargetAvgBudget=121.16 | MAE=42.75
[Warmstart 13/15] Step 220/300 | Loss=0.100581 | PredAvgBudget=112.78 |
TargetAvgBudget=120.69 | MAE=41.51
[Warmstart 13/15] Step 240/300 | Loss=0.102272 | PredAvgBudget=112.34 |
TargetAvgBudget=120.90 | MAE=41.95
[Warmstart 13/15] Step 260/300 | Loss=0.100899 | PredAvgBudget=112.97 |
TargetAvgBudget=121.43 | MAE=41.56
[Warmstart 13/15] Step 280/300 | Loss=0.099598 | PredAvgBudget=113.70 |
TargetAvgBudget=121.67 | MAE=41.44
[Warmstart 13/15] Step 300/300 | Loss=0.100591 | PredAvgBudget=113.93 |
TargetAvgBudget=121.71 | MAE=41.77
Warmstart epoch 13 summary | Loss=0.100591 | PredAvgBudget=113.93 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=119.60
  Deterministic policy eval | Done 20/100 | Acc=0.3000 | AvgBudget=118.45
  Deterministic policy eval | Done 30/100 | Acc=0.3000 | AvgBudget=116.73
  Deterministic policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=116.12
  Deterministic policy eval | Done 50/100 | Acc=0.2600 | AvgBudget=116.32
  Deterministic policy eval | Done 60/100 | Acc=0.3000 | AvgBudget=115.30
  Deterministic policy eval | Done 70/100 | Acc=0.3000 | AvgBudget=115.67
  Deterministic policy eval | Done 80/100 | Acc=0.2875 | AvgBudget=115.94
  Deterministic policy eval | Done 90/100 | Acc=0.3111 | AvgBudget=115.40
  Deterministic policy eval | Done 100/100 | Acc=0.3000 | AvgBudget=115.39
[Warmstart 14/15] Step 20/300 | Loss=0.132822 | PredAvgBudget=114.55 |
TargetAvgBudget=132.60 | MAE=46.55
[Warmstart 14/15] Step 40/300 | Loss=0.107544 | PredAvgBudget=116.90 |
TargetAvgBudget=127.90 | MAE=42.55
[Warmstart 14/15] Step 60/300 | Loss=0.098709 | PredAvgBudget=120.60 |
TargetAvgBudget=124.93 | MAE=40.63
[Warmstart 14/15] Step 80/300 | Loss=0.093570 | PredAvgBudget=116.12 |
TargetAvgBudget=119.50 | MAE=39.10
[Warmstart 14/15] Step 100/300 | Loss=0.108300 | PredAvgBudget=110.51 |
TargetAvgBudget=121.20 | MAE=40.97
[Warmstart 14/15] Step 120/300 | Loss=0.113212 | PredAvgBudget=112.45 |
TargetAvgBudget=122.60 | MAE=42.57
[Warmstart 14/15] Step 140/300 | Loss=0.112186 | PredAvgBudget=114.68 |
TargetAvgBudget=123.66 | MAE=43.04
[Warmstart 14/15] Step 160/300 | Loss=0.112658 | PredAvgBudget=115.69 |
TargetAvgBudget=122.47 | MAE=43.66
[Warmstart 14/15] Step 180/300 | Loss=0.110624 | PredAvgBudget=115.92 |
```

```
TargetAvgBudget=121.58 | MAE=43.23
[Warmstart 14/15] Step 200/300 | Loss=0.109904 | PredAvgBudget=116.83 |
TargetAvgBudget=123.02 | MAE=42.97
[Warmstart 14/15] Step 220/300 | Loss=0.105097 | PredAvgBudget=115.59 |
TargetAvgBudget=121.51 | MAE=41.52
[Warmstart 14/15] Step 240/300 | Loss=0.107909 | PredAvgBudget=113.94 |
TargetAvgBudget=121.98 | MAE=41.85
[Warmstart 14/15] Step 260/300 | Loss=0.108940 | PredAvgBudget=114.79 |
TargetAvgBudget=122.26 | MAE=42.32
[Warmstart 14/15] Step 280/300 | Loss=0.107541 | PredAvgBudget=114.24 |
TargetAvgBudget=122.09 | MAE=41.84
[Warmstart 14/15] Step 300/300 | Loss=0.107385 | PredAvgBudget=113.26 |
TargetAvgBudget=121.71 | MAE=41.48
Warmstart epoch 14 summary | Loss=0.107385 | PredAvgBudget=113.26 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=107.70
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=105.80
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=104.43
  Deterministic policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=104.28
  Deterministic policy eval | Done 50/100 | Acc=0.2800 | AvgBudget=103.86
  Deterministic policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=102.78
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=103.16
  Deterministic policy eval | Done 80/100 | Acc=0.2500 | AvgBudget=103.42
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=103.00
  Deterministic policy eval | Done 100/100 | Acc=0.3000 | AvgBudget=102.98
Saved new best warm-start policy with test avg reward = 0.5417

[Warmstart 15/15] Step 20/300 | Loss=0.077718 | PredAvgBudget=91.00 |
TargetAvgBudget=100.80 | MAE=31.20
[Warmstart 15/15] Step 40/300 | Loss=0.124150 | PredAvgBudget=86.12 |
TargetAvgBudget=111.60 | MAE=40.88
[Warmstart 15/15] Step 60/300 | Loss=0.103196 | PredAvgBudget=97.83 |
TargetAvgBudget=119.40 | MAE=37.57
[Warmstart 15/15] Step 80/300 | Loss=0.098638 | PredAvgBudget=105.44 |
TargetAvgBudget=117.90 | MAE=37.51
[Warmstart 15/15] Step 100/300 | Loss=0.102178 | PredAvgBudget=104.60 |
TargetAvgBudget=117.68 | MAE=38.58
[Warmstart 15/15] Step 120/300 | Loss=0.103805 | PredAvgBudget=106.63 |
TargetAvgBudget=119.67 | MAE=39.43
[Warmstart 15/15] Step 140/300 | Loss=0.102148 | PredAvgBudget=107.16 |
TargetAvgBudget=119.57 | MAE=39.66
[Warmstart 15/15] Step 160/300 | Loss=0.102110 | PredAvgBudget=107.99 |
TargetAvgBudget=121.00 | MAE=39.61
[Warmstart 15/15] Step 180/300 | Loss=0.103157 | PredAvgBudget=109.07 |
TargetAvgBudget=121.69 | MAE=40.17
[Warmstart 15/15] Step 200/300 | Loss=0.102058 | PredAvgBudget=108.61 |
```

```
TargetAvgBudget=120.30 | MAE=39.87
[Warmstart 15/15] Step 220/300 | Loss=0.108425 | PredAvgBudget=107.17 |
TargetAvgBudget=121.58 | MAE=40.87
[Warmstart 15/15] Step 240/300 | Loss=0.107476 | PredAvgBudget=108.26 |
TargetAvgBudget=121.98 | MAE=41.10
[Warmstart 15/15] Step 260/300 | Loss=0.104270 | PredAvgBudget=109.18 |
TargetAvgBudget=121.52 | MAE=40.32
[Warmstart 15/15] Step 280/300 | Loss=0.105747 | PredAvgBudget=109.69 |
TargetAvgBudget=121.74 | MAE=40.93
[Warmstart 15/15] Step 300/300 | Loss=0.105875 | PredAvgBudget=109.05 |
TargetAvgBudget=121.71 | MAE=40.61
Warmstart epoch 15 summary | Loss=0.105875 | PredAvgBudget=109.05 |
TargetAvgBudget=121.71

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=113.70
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=111.30
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=109.50
  Deterministic policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=108.85
  Deterministic policy eval | Done 50/100 | Acc=0.3000 | AvgBudget=108.76
  Deterministic policy eval | Done 60/100 | Acc=0.3333 | AvgBudget=107.40
  Deterministic policy eval | Done 70/100 | Acc=0.3286 | AvgBudget=107.83
  Deterministic policy eval | Done 80/100 | Acc=0.3000 | AvgBudget=108.16
  Deterministic policy eval | Done 90/100 | Acc=0.3111 | AvgBudget=107.56
  Deterministic policy eval | Done 100/100 | Acc=0.3100 | AvgBudget=107.59
Loading best warm-start policy…

Evaluating deterministic policy right after warm-start…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=107.70
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=105.80
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=104.43
  Deterministic policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=104.28
  Deterministic policy eval | Done 50/100 | Acc=0.2800 | AvgBudget=103.86
  Deterministic policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=102.78
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=103.16
  Deterministic policy eval | Done 80/100 | Acc=0.2500 | AvgBudget=103.42
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=103.00
  Deterministic policy eval | Done 100/100 | Acc=0.3000 | AvgBudget=102.98
{
  "warmstart_deterministic_eval": {
    "accuracy": 0.3,
    "avg_thinking_tokens": 102.98,
    "avg_reward": 0.5416927087709911,
    "avg_budget": 102.98,
    "min_budget": 85.0,
    "max_budget": 139.0,
    "std_budget": 11.17942753453861
  }
```

```
}

Evaluating sampled policy right after warm-start…
  Sampled policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=116.60
  Sampled policy eval | Done 20/100 | Acc=0.3000 | AvgBudget=115.25
  Sampled policy eval | Done 30/100 | Acc=0.3000 | AvgBudget=109.37
  Sampled policy eval | Done 40/100 | Acc=0.3000 | AvgBudget=108.58
  Sampled policy eval | Done 50/100 | Acc=0.3000 | AvgBudget=106.16
  Sampled policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=106.17
  Sampled policy eval | Done 70/100 | Acc=0.2857 | AvgBudget=105.91
  Sampled policy eval | Done 80/100 | Acc=0.2625 | AvgBudget=107.26
  Sampled policy eval | Done 90/100 | Acc=0.2667 | AvgBudget=108.67
  Sampled policy eval | Done 100/100 | Acc=0.2900 | AvgBudget=109.43
{
  "warmstart_sampled_eval": {
    "accuracy": 0.29,
    "avg_thinking_tokens": 109.43,
    "avg_reward": 0.5433442184678356,
    "avg_budget": 109.43,
    "min_budget": 69.0,
    "max_budget": 179.0,
    "std_budget": 27.745722192799377
  }
}

Starting RL fine-tuning…
[RL 1/5] Step 20/300 | AvgReward=0.5720 | NormRewardMean=-0.0000 | Acc=0.2500 |
AvgThinkTokens=122.80 | AvgBudget=122.80 | MinBudget=74.00 | MaxBudget=191.00 |
StdBudget=32.11
Recent budgets: [191, 91, 167, 117, 122, 87, 131, 80, 89, 74]
[RL 1/5] Step 40/300 | AvgReward=0.5627 | NormRewardMean=0.0000 | Acc=0.2500 |
AvgThinkTokens=120.72 | AvgBudget=120.72 | MinBudget=74.00 | MaxBudget=195.00 |
StdBudget=30.41
Recent budgets: [154, 138, 112, 120, 154, 108, 82, 129, 84, 142]
[RL 1/5] Step 60/300 | AvgReward=0.5969 | NormRewardMean=0.0000 | Acc=0.3000 |
AvgThinkTokens=118.28 | AvgBudget=118.28 | MinBudget=70.00 | MaxBudget=214.00 |
StdBudget=32.75
Recent budgets: [87, 93, 170, 214, 179, 70, 75, 120, 109, 76]
[RL 1/5] Step 80/300 | AvgReward=0.5530 | NormRewardMean=-0.0000 | Acc=0.2875 |
AvgThinkTokens=117.74 | AvgBudget=117.74 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=35.08
Recent budgets: [75, 79, 113, 143, 111, 165, 127, 106, 90, 169]
[RL 1/5] Step 100/300 | AvgReward=0.5703 | NormRewardMean=0.0000 | Acc=0.3200 |
AvgThinkTokens=116.64 | AvgBudget=116.64 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=34.19
Recent budgets: [89, 90, 142, 104, 100, 77, 131, 123, 166, 154]
[RL 1/5] Step 120/300 | AvgReward=0.5685 | NormRewardMean=0.0000 | Acc=0.3000 |
AvgThinkTokens=114.31 | AvgBudget=114.31 | MinBudget=70.00 | MaxBudget=217.00 |
```

```
StdBudget=33.52
Recent budgets: [101, 78, 105, 71, 101, 79, 109, 93, 76, 179]
[RL 1/5] Step 140/300 | AvgReward=0.5616 | NormRewardMean=0.0000 | Acc=0.2857 |
AvgThinkTokens=114.24 | AvgBudget=114.24 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=34.20
Recent budgets: [76, 143, 128, 177, 98, 131, 112, 78, 103, 77]
[RL 1/5] Step 160/300 | AvgReward=0.5517 | NormRewardMean=-0.0000 | Acc=0.2750 |
AvgThinkTokens=114.59 | AvgBudget=114.59 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=34.05
Recent budgets: [213, 107, 183, 132, 103, 102, 79, 105, 110, 115]
[RL 1/5] Step 180/300 | AvgReward=0.5410 | NormRewardMean=0.0000 | Acc=0.2722 |
AvgThinkTokens=114.74 | AvgBudget=114.74 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=34.11
Recent budgets: [120, 102, 108, 116, 98, 92, 108, 111, 170, 104]
[RL 1/5] Step 200/300 | AvgReward=0.5530 | NormRewardMean=0.0000 | Acc=0.2700 |
AvgThinkTokens=114.88 | AvgBudget=114.88 | MinBudget=67.00 | MaxBudget=217.00 |
StdBudget=33.73
Recent budgets: [125, 199, 125, 118, 67, 107, 104, 83, 87, 113]
[RL 1/5] Step 220/300 | AvgReward=0.5488 | NormRewardMean=0.0000 | Acc=0.2682 |
AvgThinkTokens=116.05 | AvgBudget=116.05 | MinBudget=67.00 | MaxBudget=217.00 |
StdBudget=34.14
Recent budgets: [142, 77, 163, 107, 103, 148, 82, 128, 125, 110]
[RL 1/5] Step 240/300 | AvgReward=0.5510 | NormRewardMean=-0.0000 | Acc=0.2708 |
AvgThinkTokens=115.71 | AvgBudget=115.71 | MinBudget=65.00 | MaxBudget=217.00 |
StdBudget=34.62
Recent budgets: [112, 97, 192, 141, 91, 96, 84, 191, 100, 104]
[RL 1/5] Step 260/300 | AvgReward=0.5609 | NormRewardMean=-0.0000 | Acc=0.2654 |
AvgThinkTokens=115.43 | AvgBudget=115.43 | MinBudget=65.00 | MaxBudget=217.00 |
StdBudget=35.49
Recent budgets: [69, 75, 206, 74, 216, 86, 110, 117, 160, 70]
[RL 1/5] Step 280/300 | AvgReward=0.5588 | NormRewardMean=-0.0000 | Acc=0.2679 |
AvgThinkTokens=115.04 | AvgBudget=115.04 | MinBudget=65.00 | MaxBudget=217.00 |
StdBudget=35.71
Recent budgets: [73, 117, 106, 114, 115, 89, 212, 119, 112, 124]
[RL 1/5] Step 300/300 | AvgReward=0.5573 | NormRewardMean=0.0000 | Acc=0.2667 |
AvgThinkTokens=114.13 | AvgBudget=114.13 | MinBudget=65.00 | MaxBudget=217.00 |
StdBudget=34.90
Recent budgets: [101, 127, 100, 93, 114, 110, 111, 99, 112, 85]

================================================================================
==========
RL Epoch 1 summary
Train Accuracy         : 0.2667
Train Avg ThinkTokens  : 114.13
Train Avg Reward       : 0.5573
Train Avg Budget       : 114.13
Train Min Budget       : 65.00
Train Max Budget       : 217.00
```

```
Train Std Budget      : 34.90

Deterministic policy evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=107.70
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=105.80
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=104.43
  Deterministic policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=104.28
  Deterministic policy eval | Done 50/100 | Acc=0.2800 | AvgBudget=103.86
  Deterministic policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=102.78
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=103.16
  Deterministic policy eval | Done 80/100 | Acc=0.2500 | AvgBudget=103.42
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=103.00
  Deterministic policy eval | Done 100/100 | Acc=0.3000 | AvgBudget=102.98
Test Accuracy         : 0.3000
Test Avg ThinkTokens  : 102.98
Test Avg Reward       : 0.5417
Test Avg Budget       : 102.98
Test Min Budget       : 85.00
Test Max Budget       : 139.00
Test Std Budget       : 11.18
================================================================================
==========

Saved new best RL models with deterministic test avg reward = 0.5417

[RL 2/5] Step 20/300 | AvgReward=0.4884 | NormRewardMean=0.0000 | Acc=0.2000 |
AvgThinkTokens=120.40 | AvgBudget=120.40 | MinBudget=69.00 | MaxBudget=217.00 |
StdBudget=37.20
Recent budgets: [136, 125, 196, 95, 217, 97, 130, 90, 110, 159]
[RL 2/5] Step 40/300 | AvgReward=0.5576 | NormRewardMean=0.0000 | Acc=0.2750 |
AvgThinkTokens=112.00 | AvgBudget=112.00 | MinBudget=67.00 | MaxBudget=217.00 |
StdBudget=31.20
Recent budgets: [98, 75, 141, 78, 119, 113, 113, 84, 95, 108]
[RL 2/5] Step 60/300 | AvgReward=0.5470 | NormRewardMean=-0.0000 | Acc=0.2667 |
AvgThinkTokens=112.30 | AvgBudget=112.30 | MinBudget=67.00 | MaxBudget=217.00 |
StdBudget=33.31
Recent budgets: [87, 70, 135, 79, 82, 103, 78, 112, 93, 123]
[RL 2/5] Step 80/300 | AvgReward=0.5692 | NormRewardMean=-0.0000 | Acc=0.3000 |
AvgThinkTokens=117.16 | AvgBudget=117.16 | MinBudget=67.00 | MaxBudget=217.00 |
StdBudget=32.20
Recent budgets: [117, 143, 129, 145, 144, 109, 77, 104, 135, 143]
[RL 2/5] Step 100/300 | AvgReward=0.5818 | NormRewardMean=-0.0000 | Acc=0.2900 |
AvgThinkTokens=116.82 | AvgBudget=116.82 | MinBudget=67.00 | MaxBudget=217.00 |
StdBudget=32.59
Recent budgets: [126, 120, 76, 158, 102, 187, 137, 93, 189, 83]
[RL 2/5] Step 120/300 | AvgReward=0.5911 | NormRewardMean=-0.0000 | Acc=0.2917 |
AvgThinkTokens=119.53 | AvgBudget=119.53 | MinBudget=67.00 | MaxBudget=217.00 |
StdBudget=33.60
```

```
Recent budgets: [132, 157, 94, 110, 120, 102, 117, 140, 156, 133]
[RL 2/5] Step 140/300 | AvgReward=0.5861 | NormRewardMean=-0.0000 | Acc=0.2857 |
AvgThinkTokens=118.91 | AvgBudget=118.91 | MinBudget=67.00 | MaxBudget=217.00 |
StdBudget=34.03
Recent budgets: [148, 176, 112, 75, 154, 78, 89, 89, 102, 85]
[RL 2/5] Step 160/300 | AvgReward=0.5894 | NormRewardMean=0.0000 | Acc=0.2875 |
AvgThinkTokens=117.58 | AvgBudget=117.58 | MinBudget=67.00 | MaxBudget=217.00 |
StdBudget=34.09
Recent budgets: [86, 154, 129, 81, 116, 106, 138, 68, 92, 87]
[RL 2/5] Step 180/300 | AvgReward=0.5823 | NormRewardMean=-0.0000 | Acc=0.2722 |
AvgThinkTokens=116.48 | AvgBudget=116.48 | MinBudget=67.00 | MaxBudget=217.00 |
StdBudget=33.71
Recent budgets: [112, 94, 95, 79, 137, 81, 133, 86, 132, 103]
[RL 2/5] Step 200/300 | AvgReward=0.5793 | NormRewardMean=-0.0000 | Acc=0.2650 |
AvgThinkTokens=115.97 | AvgBudget=115.97 | MinBudget=67.00 | MaxBudget=217.00 |
StdBudget=33.51
Recent budgets: [84, 101, 120, 182, 85, 148, 69, 124, 80, 86]
[RL 2/5] Step 220/300 | AvgReward=0.5676 | NormRewardMean=-0.0000 | Acc=0.2545 |
AvgThinkTokens=117.25 | AvgBudget=117.25 | MinBudget=67.00 | MaxBudget=217.00 |
StdBudget=34.08
Recent budgets: [116, 136, 102, 198, 156, 155, 91, 71, 152, 91]
[RL 2/5] Step 240/300 | AvgReward=0.5687 | NormRewardMean=-0.0000 | Acc=0.2500 |
AvgThinkTokens=116.11 | AvgBudget=116.11 | MinBudget=67.00 | MaxBudget=217.00 |
StdBudget=34.05
Recent budgets: [97, 81, 94, 87, 86, 74, 124, 84, 151, 98]
[RL 2/5] Step 260/300 | AvgReward=0.5747 | NormRewardMean=-0.0000 | Acc=0.2615 |
AvgThinkTokens=116.73 | AvgBudget=116.73 | MinBudget=67.00 | MaxBudget=220.00 |
StdBudget=35.10
Recent budgets: [207, 69, 163, 136, 84, 67, 220, 85, 108, 109]
[RL 2/5] Step 280/300 | AvgReward=0.5736 | NormRewardMean=-0.0000 | Acc=0.2643 |
AvgThinkTokens=117.16 | AvgBudget=117.16 | MinBudget=67.00 | MaxBudget=220.00 |
StdBudget=35.83
Recent budgets: [200, 90, 189, 176, 82, 102, 119, 159, 80, 93]
[RL 2/5] Step 300/300 | AvgReward=0.5731 | NormRewardMean=-0.0000 | Acc=0.2600 |
AvgThinkTokens=116.55 | AvgBudget=116.55 | MinBudget=67.00 | MaxBudget=220.00 |
StdBudget=35.69
Recent budgets: [105, 127, 91, 71, 72, 88, 105, 85, 123, 90]

================================================================================
==========
RL Epoch 2 summary
Train Accuracy        : 0.2600
Train Avg ThinkTokens : 116.55
Train Avg Reward      : 0.5731
Train Avg Budget      : 116.55
Train Min Budget      : 67.00
Train Max Budget      : 220.00
Train Std Budget      : 35.69
```

```
Deterministic policy evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=107.70
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=105.80
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=104.43
  Deterministic policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=104.28
  Deterministic policy eval | Done 50/100 | Acc=0.2800 | AvgBudget=103.86
  Deterministic policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=102.78
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=103.16
  Deterministic policy eval | Done 80/100 | Acc=0.2500 | AvgBudget=103.42
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=103.00
  Deterministic policy eval | Done 100/100 | Acc=0.3000 | AvgBudget=102.98
Test Accuracy         : 0.3000
Test Avg ThinkTokens  : 102.98
Test Avg Reward       : 0.5417
Test Avg Budget       : 102.98
Test Min Budget       : 85.00
Test Max Budget       : 139.00
Test Std Budget       : 11.18
================================================================================
==========

[RL 3/5] Step 20/300 | AvgReward=0.4964 | NormRewardMean=0.0000 | Acc=0.1500 |
AvgThinkTokens=104.10 | AvgBudget=104.10 | MinBudget=74.00 | MaxBudget=175.00 |
StdBudget=26.31
Recent budgets: [100, 114, 74, 75, 104, 118, 114, 86, 126, 167]
[RL 3/5] Step 40/300 | AvgReward=0.5117 | NormRewardMean=0.0000 | Acc=0.1750 |
AvgThinkTokens=103.15 | AvgBudget=103.15 | MinBudget=66.00 | MaxBudget=175.00 |
StdBudget=29.03
Recent budgets: [92, 83, 82, 101, 73, 90, 69, 112, 83, 80]
[RL 3/5] Step 60/300 | AvgReward=0.5089 | NormRewardMean=-0.0000 | Acc=0.2167 |
AvgThinkTokens=103.03 | AvgBudget=103.03 | MinBudget=66.00 | MaxBudget=175.00 |
StdBudget=27.68
Recent budgets: [131, 102, 114, 84, 77, 95, 86, 97, 101, 98]
[RL 3/5] Step 80/300 | AvgReward=0.4763 | NormRewardMean=-0.0000 | Acc=0.1875 |
AvgThinkTokens=104.66 | AvgBudget=104.66 | MinBudget=66.00 | MaxBudget=175.00 |
StdBudget=27.38
Recent budgets: [91, 161, 95, 144, 89, 124, 110, 97, 91, 158]
[RL 3/5] Step 100/300 | AvgReward=0.4758 | NormRewardMean=-0.0000 | Acc=0.1900 |
AvgThinkTokens=105.05 | AvgBudget=105.05 | MinBudget=66.00 | MaxBudget=180.00 |
StdBudget=27.87
Recent budgets: [157, 78, 109, 98, 140, 94, 75, 79, 180, 70]
[RL 3/5] Step 120/300 | AvgReward=0.5185 | NormRewardMean=-0.0000 | Acc=0.2417 |
AvgThinkTokens=107.80 | AvgBudget=107.80 | MinBudget=66.00 | MaxBudget=193.00 |
StdBudget=28.41
Recent budgets: [103, 79, 135, 129, 133, 137, 94, 144, 95, 134]
[RL 3/5] Step 140/300 | AvgReward=0.5338 | NormRewardMean=0.0000 | Acc=0.2500 |
AvgThinkTokens=107.59 | AvgBudget=107.59 | MinBudget=66.00 | MaxBudget=193.00 |
```

```
StdBudget=28.41
Recent budgets: [138, 108, 77, 109, 111, 183, 127, 104, 91, 143]
[RL 3/5] Step 160/300 | AvgReward=0.5269 | NormRewardMean=-0.0000 | Acc=0.2313 |
AvgThinkTokens=107.58 | AvgBudget=107.58 | MinBudget=66.00 | MaxBudget=193.00 |
StdBudget=28.13
Recent budgets: [84, 89, 119, 89, 132, 89, 78, 97, 135, 88]
[RL 3/5] Step 180/300 | AvgReward=0.5226 | NormRewardMean=0.0000 | Acc=0.2333 |
AvgThinkTokens=108.07 | AvgBudget=108.07 | MinBudget=66.00 | MaxBudget=193.00 |
StdBudget=28.31
Recent budgets: [146, 100, 93, 82, 173, 87, 130, 87, 97, 76]
[RL 3/5] Step 200/300 | AvgReward=0.5303 | NormRewardMean=0.0000 | Acc=0.2500 |
AvgThinkTokens=108.65 | AvgBudget=108.65 | MinBudget=66.00 | MaxBudget=193.00 |
StdBudget=28.44
Recent budgets: [69, 73, 99, 112, 88, 79, 139, 130, 83, 114]
[RL 3/5] Step 220/300 | AvgReward=0.5235 | NormRewardMean=-0.0000 | Acc=0.2455 |
AvgThinkTokens=108.75 | AvgBudget=108.75 | MinBudget=65.00 | MaxBudget=196.00 |
StdBudget=29.30
Recent budgets: [96, 99, 92, 126, 76, 76, 139, 127, 71, 94]
[RL 3/5] Step 240/300 | AvgReward=0.5236 | NormRewardMean=-0.0000 | Acc=0.2333 |
AvgThinkTokens=109.40 | AvgBudget=109.40 | MinBudget=65.00 | MaxBudget=200.00 |
StdBudget=29.63
Recent budgets: [200, 136, 85, 154, 111, 123, 81, 125, 167, 98]
[RL 3/5] Step 260/300 | AvgReward=0.5299 | NormRewardMean=-0.0000 | Acc=0.2462 |
AvgThinkTokens=109.05 | AvgBudget=109.05 | MinBudget=65.00 | MaxBudget=200.00 |
StdBudget=29.59
Recent budgets: [106, 74, 96, 154, 70, 93, 133, 79, 74, 82]
[RL 3/5] Step 280/300 | AvgReward=0.5316 | NormRewardMean=0.0000 | Acc=0.2464 |
AvgThinkTokens=109.40 | AvgBudget=109.40 | MinBudget=65.00 | MaxBudget=200.00 |
StdBudget=29.51
Recent budgets: [98, 117, 83, 111, 175, 113, 83, 73, 138, 108]
[RL 3/5] Step 300/300 | AvgReward=0.5392 | NormRewardMean=-0.0000 | Acc=0.2600 |
AvgThinkTokens=109.70 | AvgBudget=109.70 | MinBudget=65.00 | MaxBudget=200.00 |
StdBudget=29.39
Recent budgets: [144, 162, 108, 71, 116, 134, 100, 85, 99, 119]

================================================================================
==========
RL Epoch 3 summary
Train Accuracy         : 0.2600
Train Avg ThinkTokens  : 109.70
Train Avg Reward       : 0.5392
Train Avg Budget       : 109.70
Train Min Budget       : 65.00
Train Max Budget       : 200.00
Train Std Budget       : 29.39

Deterministic policy evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=107.70
```

```
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=105.80
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=104.43
  Deterministic policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=104.28
  Deterministic policy eval | Done 50/100 | Acc=0.2800 | AvgBudget=103.86
  Deterministic policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=102.78
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=103.16
  Deterministic policy eval | Done 80/100 | Acc=0.2500 | AvgBudget=103.42
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=103.00
  Deterministic policy eval | Done 100/100 | Acc=0.3000 | AvgBudget=102.98
Test Accuracy         : 0.3000
Test Avg ThinkTokens  : 102.98
Test Avg Reward       : 0.5417
Test Avg Budget       : 102.98
Test Min Budget       : 85.00
Test Max Budget       : 139.00
Test Std Budget       : 11.18
================================================================================
==========

[RL 4/5] Step 20/300 | AvgReward=0.5797 | NormRewardMean=-0.0000 | Acc=0.2000 |
AvgThinkTokens=119.65 | AvgBudget=119.65 | MinBudget=70.00 | MaxBudget=182.00 |
StdBudget=35.55
Recent budgets: [168, 128, 80, 70, 96, 81, 87, 168, 111, 138]
[RL 4/5] Step 40/300 | AvgReward=0.6086 | NormRewardMean=-0.0000 | Acc=0.3000 |
AvgThinkTokens=111.42 | AvgBudget=111.42 | MinBudget=70.00 | MaxBudget=205.00 |
StdBudget=35.59
Recent budgets: [85, 97, 84, 83, 78, 155, 103, 90, 85, 102]
[RL 4/5] Step 60/300 | AvgReward=0.5403 | NormRewardMean=-0.0000 | Acc=0.2000 |
AvgThinkTokens=110.70 | AvgBudget=110.70 | MinBudget=69.00 | MaxBudget=205.00 |
StdBudget=33.26
Recent budgets: [114, 143, 183, 125, 69, 73, 93, 118, 75, 104]
[RL 4/5] Step 80/300 | AvgReward=0.5578 | NormRewardMean=-0.0000 | Acc=0.2125 |
AvgThinkTokens=109.97 | AvgBudget=109.97 | MinBudget=69.00 | MaxBudget=205.00 |
StdBudget=32.27
Recent budgets: [93, 141, 97, 75, 69, 109, 77, 127, 110, 111]
[RL 4/5] Step 100/300 | AvgReward=0.5636 | NormRewardMean=0.0000 | Acc=0.2200 |
AvgThinkTokens=109.26 | AvgBudget=109.26 | MinBudget=69.00 | MaxBudget=205.00 |
StdBudget=31.64
Recent budgets: [114, 100, 110, 79, 81, 82, 105, 118, 120, 97]
[RL 4/5] Step 120/300 | AvgReward=0.5863 | NormRewardMean=-0.0000 | Acc=0.2500 |
AvgThinkTokens=111.17 | AvgBudget=111.17 | MinBudget=69.00 | MaxBudget=205.00 |
StdBudget=32.28
Recent budgets: [91, 135, 74, 150, 143, 92, 190, 100, 142, 141]
[RL 4/5] Step 140/300 | AvgReward=0.5878 | NormRewardMean=-0.0000 | Acc=0.2571 |
AvgThinkTokens=110.81 | AvgBudget=110.81 | MinBudget=69.00 | MaxBudget=205.00 |
StdBudget=31.21
Recent budgets: [97, 116, 88, 131, 75, 116, 78, 87, 150, 153]
[RL 4/5] Step 160/300 | AvgReward=0.5904 | NormRewardMean=-0.0000 | Acc=0.2750 |
```

```
AvgThinkTokens=111.56 | AvgBudget=111.56 | MinBudget=69.00 | MaxBudget=205.00 |
StdBudget=31.47
Recent budgets: [78, 79, 126, 112, 114, 132, 110, 78, 169, 97]
[RL 4/5] Step 180/300 | AvgReward=0.5561 | NormRewardMean=-0.0000 | Acc=0.2611 |
AvgThinkTokens=111.24 | AvgBudget=111.24 | MinBudget=69.00 | MaxBudget=205.00 |
StdBudget=31.02
Recent budgets: [80, 69, 92, 154, 117, 135, 164, 128, 144, 93]
[RL 4/5] Step 200/300 | AvgReward=0.5625 | NormRewardMean=-0.0000 | Acc=0.2600 |
AvgThinkTokens=110.59 | AvgBudget=110.59 | MinBudget=68.00 | MaxBudget=205.00 |
StdBudget=30.74
Recent budgets: [153, 73, 123, 89, 157, 133, 102, 80, 106, 123]
[RL 4/5] Step 220/300 | AvgReward=0.5582 | NormRewardMean=-0.0000 | Acc=0.2409 |
AvgThinkTokens=110.39 | AvgBudget=110.39 | MinBudget=68.00 | MaxBudget=205.00 |
StdBudget=30.59
Recent budgets: [136, 101, 136, 101, 108, 106, 88, 123, 185, 100]
[RL 4/5] Step 240/300 | AvgReward=0.5602 | NormRewardMean=-0.0000 | Acc=0.2375 |
AvgThinkTokens=110.40 | AvgBudget=110.40 | MinBudget=68.00 | MaxBudget=209.00 |
StdBudget=30.85
Recent budgets: [146, 102, 99, 77, 73, 146, 70, 79, 100, 84]
[RL 4/5] Step 260/300 | AvgReward=0.5565 | NormRewardMean=0.0000 | Acc=0.2346 |
AvgThinkTokens=111.08 | AvgBudget=111.08 | MinBudget=68.00 | MaxBudget=209.00 |
StdBudget=31.38
Recent budgets: [98, 172, 112, 84, 102, 76, 170, 85, 143, 169]
[RL 4/5] Step 280/300 | AvgReward=0.5531 | NormRewardMean=0.0000 | Acc=0.2357 |
AvgThinkTokens=110.85 | AvgBudget=110.85 | MinBudget=68.00 | MaxBudget=209.00 |
StdBudget=31.00
Recent budgets: [116, 82, 132, 106, 100, 113, 119, 90, 87, 116]
[RL 4/5] Step 300/300 | AvgReward=0.5489 | NormRewardMean=-0.0000 | Acc=0.2367 |
AvgThinkTokens=111.49 | AvgBudget=111.49 | MinBudget=68.00 | MaxBudget=209.00 |
StdBudget=31.23
Recent budgets: [86, 125, 83, 79, 165, 152, 143, 119, 87, 107]

================================================================================
==========
RL Epoch 4 summary
Train Accuracy         : 0.2367
Train Avg ThinkTokens  : 111.49
Train Avg Reward       : 0.5489
Train Avg Budget       : 111.49
Train Min Budget       : 68.00
Train Max Budget       : 209.00
Train Std Budget       : 31.23

Deterministic policy evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=107.70
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=105.80
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=104.43
  Deterministic policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=104.28
```

```
  Deterministic policy eval | Done 50/100 | Acc=0.2800 | AvgBudget=103.86
  Deterministic policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=102.78
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=103.16
  Deterministic policy eval | Done 80/100 | Acc=0.2500 | AvgBudget=103.42
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=103.00
  Deterministic policy eval | Done 100/100 | Acc=0.3000 | AvgBudget=102.98
Test Accuracy         : 0.3000
Test Avg ThinkTokens  : 102.98
Test Avg Reward       : 0.5417
Test Avg Budget       : 102.98
Test Min Budget       : 85.00
Test Max Budget       : 139.00
Test Std Budget       : 11.18
================================================================================
==========

[RL 5/5] Step 20/300 | AvgReward=0.5372 | NormRewardMean=0.0000 | Acc=0.2000 |
AvgThinkTokens=105.60 | AvgBudget=105.60 | MinBudget=69.00 | MaxBudget=151.00 |
StdBudget=27.99
Recent budgets: [69, 98, 106, 150, 105, 83, 143, 151, 99, 70]
[RL 5/5] Step 40/300 | AvgReward=0.5600 | NormRewardMean=-0.0000 | Acc=0.2750 |
AvgThinkTokens=106.20 | AvgBudget=106.20 | MinBudget=69.00 | MaxBudget=166.00 |
StdBudget=27.84
Recent budgets: [144, 131, 112, 87, 118, 72, 88, 98, 73, 72]
[RL 5/5] Step 60/300 | AvgReward=0.5101 | NormRewardMean=-0.0000 | Acc=0.2167 |
AvgThinkTokens=102.93 | AvgBudget=102.93 | MinBudget=69.00 | MaxBudget=166.00 |
StdBudget=26.41
Recent budgets: [79, 85, 102, 144, 111, 78, 72, 109, 139, 124]
[RL 5/5] Step 80/300 | AvgReward=0.5276 | NormRewardMean=-0.0000 | Acc=0.2500 |
AvgThinkTokens=103.85 | AvgBudget=103.85 | MinBudget=67.00 | MaxBudget=197.00 |
StdBudget=30.13
Recent budgets: [76, 107, 194, 108, 151, 89, 88, 80, 75, 122]
[RL 5/5] Step 100/300 | AvgReward=0.5205 | NormRewardMean=-0.0000 | Acc=0.2500 |
AvgThinkTokens=106.21 | AvgBudget=106.21 | MinBudget=67.00 | MaxBudget=197.00 |
StdBudget=29.29
Recent budgets: [119, 103, 98, 88, 154, 97, 125, 146, 116, 138]
[RL 5/5] Step 120/300 | AvgReward=0.4966 | NormRewardMean=0.0000 | Acc=0.2333 |
AvgThinkTokens=106.07 | AvgBudget=106.07 | MinBudget=67.00 | MaxBudget=197.00 |
StdBudget=28.18
Recent budgets: [82, 91, 78, 109, 132, 89, 88, 114, 94, 93]
[RL 5/5] Step 140/300 | AvgReward=0.5141 | NormRewardMean=-0.0000 | Acc=0.2214 |
AvgThinkTokens=106.81 | AvgBudget=106.81 | MinBudget=67.00 | MaxBudget=197.00 |
StdBudget=28.75
Recent budgets: [74, 90, 157, 74, 141, 113, 135, 93, 89, 77]
[RL 5/5] Step 160/300 | AvgReward=0.5147 | NormRewardMean=0.0000 | Acc=0.2188 |
AvgThinkTokens=107.85 | AvgBudget=107.85 | MinBudget=67.00 | MaxBudget=197.00 |
StdBudget=29.83
Recent budgets: [92, 138, 71, 100, 73, 85, 159, 163, 86, 163]
```

```
[RL 5/5] Step 180/300 | AvgReward=0.5122 | NormRewardMean=-0.0000 | Acc=0.2278 |
AvgThinkTokens=108.27 | AvgBudget=108.27 | MinBudget=67.00 | MaxBudget=197.00 |
StdBudget=30.12
Recent budgets: [77, 89, 78, 134, 71, 80, 101, 108, 72, 116]
[RL 5/5] Step 200/300 | AvgReward=0.5157 | NormRewardMean=0.0000 | Acc=0.2250 |
AvgThinkTokens=108.73 | AvgBudget=108.73 | MinBudget=67.00 | MaxBudget=197.00 |
StdBudget=30.64
Recent budgets: [84, 87, 79, 93, 144, 70, 155, 152, 90, 80]
[RL 5/5] Step 220/300 | AvgReward=0.5036 | NormRewardMean=0.0000 | Acc=0.2182 |
AvgThinkTokens=108.43 | AvgBudget=108.43 | MinBudget=67.00 | MaxBudget=197.00 |
StdBudget=30.20
Recent budgets: [164, 109, 124, 106, 81, 91, 90, 84, 157, 84]
[RL 5/5] Step 240/300 | AvgReward=0.5041 | NormRewardMean=-0.0000 | Acc=0.2250 |
AvgThinkTokens=107.87 | AvgBudget=107.87 | MinBudget=67.00 | MaxBudget=197.00 |
StdBudget=30.06
Recent budgets: [103, 96, 87, 96, 186, 75, 78, 105, 69, 159]
[RL 5/5] Step 260/300 | AvgReward=0.5053 | NormRewardMean=-0.0000 | Acc=0.2192 |
AvgThinkTokens=107.69 | AvgBudget=107.69 | MinBudget=67.00 | MaxBudget=197.00 |
StdBudget=29.31
Recent budgets: [109, 109, 100, 103, 108, 75, 130, 117, 119, 138]
[RL 5/5] Step 280/300 | AvgReward=0.5094 | NormRewardMean=-0.0000 | Acc=0.2250 |
AvgThinkTokens=107.23 | AvgBudget=107.23 | MinBudget=67.00 | MaxBudget=197.00 |
StdBudget=28.82
Recent budgets: [88, 100, 148, 141, 117, 76, 92, 95, 106, 84]
[RL 5/5] Step 300/300 | AvgReward=0.5014 | NormRewardMean=0.0000 | Acc=0.2200 |
AvgThinkTokens=107.48 | AvgBudget=107.48 | MinBudget=67.00 | MaxBudget=197.00 |
StdBudget=29.24
Recent budgets: [72, 190, 67, 99, 93, 125, 107, 112, 92, 133]

================================================================================
==========
RL Epoch 5 summary
Train Accuracy        : 0.2200
Train Avg ThinkTokens : 107.48
Train Avg Reward      : 0.5014
Train Avg Budget      : 107.48
Train Min Budget      : 67.00
Train Max Budget      : 197.00
Train Std Budget      : 29.24

Deterministic policy evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=107.70
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=105.80
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=104.43
  Deterministic policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=104.28
  Deterministic policy eval | Done 50/100 | Acc=0.2800 | AvgBudget=103.86
  Deterministic policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=102.78
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=103.16
```

```
  Deterministic policy eval | Done 80/100 | Acc=0.2500 | AvgBudget=103.42
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=103.00
  Deterministic policy eval | Done 100/100 | Acc=0.3000 | AvgBudget=102.98
Test Accuracy         : 0.3000
Test Avg ThinkTokens  : 102.98
Test Avg Reward       : 0.5417
Test Avg Budget       : 102.98
Test Min Budget       : 85.00
Test Max Budget       : 139.00
Test Std Budget       : 11.18
================================================================================
==========

Loading best saved policy/value for final evaluation…

Final deterministic policy evaluation…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=107.70
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=105.80
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=104.43
  Deterministic policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=104.28
  Deterministic policy eval | Done 50/100 | Acc=0.2800 | AvgBudget=103.86
  Deterministic policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=102.78
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=103.16
  Deterministic policy eval | Done 80/100 | Acc=0.2500 | AvgBudget=103.42
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=103.00
  Deterministic policy eval | Done 100/100 | Acc=0.3000 | AvgBudget=102.98
{
  "final_deterministic_eval": {
    "accuracy": 0.3,
    "avg_thinking_tokens": 102.98,
    "avg_reward": 0.5416927087709911,
    "avg_budget": 102.98,
    "min_budget": 85.0,
    "max_budget": 139.0,
    "std_budget": 11.17942753453861
  }
}

Final sampled policy evaluation…
  Sampled policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=105.20
  Sampled policy eval | Done 20/100 | Acc=0.3000 | AvgBudget=112.35
  Sampled policy eval | Done 30/100 | Acc=0.3000 | AvgBudget=108.60
  Sampled policy eval | Done 40/100 | Acc=0.3000 | AvgBudget=111.55
  Sampled policy eval | Done 50/100 | Acc=0.3000 | AvgBudget=108.88
  Sampled policy eval | Done 60/100 | Acc=0.3167 | AvgBudget=107.62
  Sampled policy eval | Done 70/100 | Acc=0.3000 | AvgBudget=108.66
  Sampled policy eval | Done 80/100 | Acc=0.2750 | AvgBudget=107.50
  Sampled policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=107.28
```

```
  Sampled policy eval | Done 100/100 | Acc=0.2900 | AvgBudget=107.50
{
  "final_sampled_eval": {
    "accuracy": 0.29,
    "avg_thinking_tokens": 107.5,
    "avg_reward": 0.5135256577060494,
    "avg_budget": 107.5,
    "min_budget": 68.0,
    "max_budget": 177.0,
    "std_budget": 26.206296953213364
  }
}

Saved final policy to two_stage_continuous_budget_policy_v8_final.pt
Saved final value net to two_stage_value_net_v8_final.pt
```

[2]:
```
!pip install numpy torch datasets transformers sentence-transformers
```

```
Collecting numpy
  Downloading
numpy-2.4.3-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata
(6.6 kB)
Collecting torch
  Downloading torch-2.10.0-3-cp313-cp313-manylinux_2_28_x86_64.whl.metadata (31
kB)
Collecting datasets
  Using cached datasets-4.8.2-py3-none-any.whl.metadata (19 kB)
Collecting transformers
  Using cached transformers-5.3.0-py3-none-any.whl.metadata (32 kB)
Collecting sentence-transformers
  Using cached sentence_transformers-5.3.0-py3-none-any.whl.metadata (16 kB)
Collecting filelock (from torch)
  Downloading filelock-3.25.2-py3-none-any.whl.metadata (2.0 kB)
Requirement already satisfied: typing-extensions>=4.10.0 in
/opt/conda/lib/python3.13/site-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /opt/conda/lib/python3.13/site-
packages (from torch) (80.9.0)
Collecting sympy>=1.13.3 (from torch)
  Downloading sympy-1.14.0-py3-none-any.whl.metadata (12 kB)
Collecting networkx>=2.5.1 (from torch)
  Downloading networkx-3.6.1-py3-none-any.whl.metadata (6.8 kB)
Requirement already satisfied: jinja2 in /opt/conda/lib/python3.13/site-packages
(from torch) (3.1.6)
Collecting fsspec>=0.8.5 (from torch)
  Downloading fsspec-2026.2.0-py3-none-any.whl.metadata (10 kB)
Collecting cuda-bindings==12.9.4 (from torch)
  Downloading cuda_bindings-12.9.4-cp313-cp313-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (2.6 kB)
```

```
Collecting nvidia-cuda-nvrtc-cu12==12.8.93 (from torch)
  Downloading nvidia_cuda_nvrtc_cu12-12.8.93-py3-none-
manylinux2010_x86_64.manylinux_2_12_x86_64.whl.metadata (1.7 kB)
Collecting nvidia-cuda-runtime-cu12==12.8.90 (from torch)
  Downloading nvidia_cuda_runtime_cu12-12.8.90-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (1.7 kB)
Collecting nvidia-cuda-cupti-cu12==12.8.90 (from torch)
  Downloading nvidia_cuda_cupti_cu12-12.8.90-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (1.7 kB)
Collecting nvidia-cudnn-cu12==9.10.2.21 (from torch)
  Downloading nvidia_cudnn_cu12-9.10.2.21-py3-none-
manylinux_2_27_x86_64.whl.metadata (1.8 kB)
Collecting nvidia-cublas-cu12==12.8.4.1 (from torch)
  Downloading nvidia_cublas_cu12-12.8.4.1-py3-none-
manylinux_2_27_x86_64.whl.metadata (1.7 kB)
Collecting nvidia-cufft-cu12==11.3.3.83 (from torch)
  Downloading nvidia_cufft_cu12-11.3.3.83-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (1.7 kB)
Collecting nvidia-curand-cu12==10.3.9.90 (from torch)
  Downloading nvidia_curand_cu12-10.3.9.90-py3-none-
manylinux_2_27_x86_64.whl.metadata (1.7 kB)
Collecting nvidia-cusolver-cu12==11.7.3.90 (from torch)
  Downloading nvidia_cusolver_cu12-11.7.3.90-py3-none-
manylinux_2_27_x86_64.whl.metadata (1.8 kB)
Collecting nvidia-cusparse-cu12==12.5.8.93 (from torch)
  Downloading nvidia_cusparse_cu12-12.5.8.93-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (1.8 kB)
Collecting nvidia-cusparselt-cu12==0.7.1 (from torch)
  Downloading nvidia_cusparselt_cu12-0.7.1-py3-none-
manylinux2014_x86_64.whl.metadata (7.0 kB)
Collecting nvidia-nccl-cu12==2.27.5 (from torch)
  Downloading nvidia_nccl_cu12-2.27.5-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (2.0 kB)
Collecting nvidia-nvshmem-cu12==3.4.5 (from torch)
  Downloading nvidia_nvshmem_cu12-3.4.5-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (2.1 kB)
Collecting nvidia-nvtx-cu12==12.8.90 (from torch)
  Downloading nvidia_nvtx_cu12-12.8.90-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (1.8 kB)
Collecting nvidia-nvjitlink-cu12==12.8.93 (from torch)
  Downloading nvidia_nvjitlink_cu12-12.8.93-py3-none-
manylinux2010_x86_64.manylinux_2_12_x86_64.whl.metadata (1.7 kB)
Collecting nvidia-cufile-cu12==1.13.1.3 (from torch)
  Downloading nvidia_cufile_cu12-1.13.1.3-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (1.7 kB)
Collecting triton==3.6.0 (from torch)
  Downloading triton-3.6.0-cp313-cp313-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (1.7 kB)
```

```
Collecting cuda-pathfinder~=1.1 (from cuda-bindings==12.9.4->torch)
  Downloading cuda_pathfinder-1.4.3-py3-none-any.whl.metadata (1.9 kB)
Collecting pyarrow>=21.0.0 (from datasets)
  Downloading pyarrow-23.0.1-cp313-cp313-manylinux_2_28_x86_64.whl.metadata (3.1
kB)
Collecting dill<0.4.2,>=0.3.0 (from datasets)
  Using cached dill-0.4.1-py3-none-any.whl.metadata (10 kB)
Collecting pandas (from datasets)
  Downloading pandas-3.0.1-cp313-cp313-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
Requirement already satisfied: requests>=2.32.2 in
/opt/conda/lib/python3.13/site-packages (from datasets) (2.32.5)
Requirement already satisfied: httpx<1.0.0 in /opt/conda/lib/python3.13/site-
packages (from datasets) (0.28.1)
Requirement already satisfied: tqdm>=4.66.3 in /opt/conda/lib/python3.13/site-
packages (from datasets) (4.67.1)
Collecting xxhash (from datasets)
  Using cached xxhash-3.6.0-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(13 kB)
Collecting multiprocess<0.70.20 (from datasets)
  Using cached multiprocess-0.70.19-py313-none-any.whl.metadata (7.5 kB)
Collecting huggingface-hub<2.0,>=0.25.0 (from datasets)
  Using cached huggingface_hub-1.7.1-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: packaging in /opt/conda/lib/python3.13/site-
packages (from datasets) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /opt/conda/lib/python3.13/site-
packages (from datasets) (6.0.3)
Collecting aiohttp!=4.0.0a0,!=4.0.0a1 (from
fsspec[http]<=2026.2.0,>=2023.1.0->datasets)
  Downloading aiohttp-3.13.3-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(8.1 kB)
Requirement already satisfied: anyio in /opt/conda/lib/python3.13/site-packages
(from httpx<1.0.0->datasets) (4.12.0)
Requirement already satisfied: certifi in /opt/conda/lib/python3.13/site-
packages (from httpx<1.0.0->datasets) (2026.1.4)
Requirement already satisfied: httpcore==1.* in /opt/conda/lib/python3.13/site-
packages (from httpx<1.0.0->datasets) (1.0.9)
Requirement already satisfied: idna in /opt/conda/lib/python3.13/site-packages
(from httpx<1.0.0->datasets) (3.11)
Requirement already satisfied: h11>=0.16 in /opt/conda/lib/python3.13/site-
packages (from httpcore==1.*->httpx<1.0.0->datasets) (0.16.0)
Collecting hf-xet<2.0.0,>=1.4.2 (from huggingface-hub<2.0,>=0.25.0->datasets)
  Using cached
hf_xet-1.4.2-cp37-abi3-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata
(4.9 kB)
Collecting typer (from huggingface-hub<2.0,>=0.25.0->datasets)
```

```
  Using cached typer-0.24.1-py3-none-any.whl.metadata (16 kB)
Collecting regex!=2019.12.17 (from transformers)
  Using cached regex-2026.2.28-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(40 kB)
Collecting tokenizers<=0.23.0,>=0.22.0 (from transformers)
  Using cached tokenizers-0.22.2-cp39-abi3-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (7.3 kB)
Collecting safetensors>=0.4.3 (from transformers)
  Using cached safetensors-0.7.0-cp38-abi3-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.1 kB)
Collecting scikit-learn (from sentence-transformers)
  Downloading scikit_learn-1.8.0-cp313-cp313-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (11 kB)
Collecting scipy (from sentence-transformers)
  Downloading scipy-1.17.1-cp313-cp313-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (62 kB)
Collecting aiohappyeyeballs>=2.5.0 (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets)
  Downloading aiohappyeyeballs-2.6.1-py3-none-any.whl.metadata (5.9 kB)
Collecting aiosignal>=1.4.0 (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets)
  Downloading aiosignal-1.4.0-py3-none-any.whl.metadata (3.7 kB)
Requirement already satisfied: attrs>=17.3.0 in /opt/conda/lib/python3.13/site-
packages (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets)
(25.4.0)
Collecting frozenlist>=1.1.1 (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets)
  Downloading frozenlist-1.8.0-cp313-cp313-
manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (20
kB)
Collecting multidict<7.0,>=4.5 (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets)
  Downloading multidict-6.7.1-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(5.3 kB)
Collecting propcache>=0.2.0 (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets)
  Downloading propcache-0.4.1-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(13 kB)
Collecting yarl<2.0,>=1.17.0 (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets)
  Downloading yarl-1.23.0-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(79 kB)
Requirement already satisfied: charset_normalizer<4,>=2 in
```

```
/opt/conda/lib/python3.13/site-packages (from requests>=2.32.2->datasets)
(3.4.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/opt/conda/lib/python3.13/site-packages (from requests>=2.32.2->datasets)
(2.6.2)
Collecting mpmath<1.4,>=1.1.0 (from sympy>=1.13.3->torch)
  Downloading mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: MarkupSafe>=2.0 in
/opt/conda/lib/python3.13/site-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: python-dateutil>=2.8.2 in
/opt/conda/lib/python3.13/site-packages (from pandas->datasets) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.13/site-
packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)
Collecting joblib>=1.3.0 (from scikit-learn->sentence-transformers)
  Downloading joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting threadpoolctl>=3.2.0 (from scikit-learn->sentence-transformers)
  Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Collecting click>=8.2.1 (from typer->huggingface-hub<2.0,>=0.25.0->datasets)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting shellingham>=1.3.0 (from typer->huggingface-
hub<2.0,>=0.25.0->datasets)
  Using cached shellingham-1.5.4-py2.py3-none-any.whl.metadata (3.5 kB)
Collecting rich>=12.3.0 (from typer->huggingface-hub<2.0,>=0.25.0->datasets)
  Using cached rich-14.3.3-py3-none-any.whl.metadata (18 kB)
Collecting annotated-doc>=0.0.2 (from typer->huggingface-
hub<2.0,>=0.25.0->datasets)
  Using cached annotated_doc-0.0.4-py3-none-any.whl.metadata (6.6 kB)
Collecting markdown-it-py>=2.2.0 (from rich>=12.3.0->typer->huggingface-
hub<2.0,>=0.25.0->datasets)
  Using cached markdown_it_py-4.0.0-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/opt/conda/lib/python3.13/site-packages (from rich>=12.3.0->typer->huggingface-
hub<2.0,>=0.25.0->datasets) (2.19.2)
Collecting mdurl~=0.1 (from markdown-it-
py>=2.2.0->rich>=12.3.0->typer->huggingface-hub<2.0,>=0.25.0->datasets)
  Using cached mdurl-0.1.2-py3-none-any.whl.metadata (1.6 kB)
Downloading
numpy-2.4.3-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.6
MB)
                        16.6/16.6 MB
90.4 MB/s  0:00:00
Downloading torch-2.10.0-3-cp313-cp313-manylinux_2_28_x86_64.whl (915.6
MB)
                        915.6/915.6 MB
44.5 MB/s  0:00:08m0:00:0100:01
Downloading
cuda_bindings-12.9.4-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl
(11.9 MB)
```

```
                         11.9/11.9 MB
87.8 MB/s  0:00:00
Downloading nvidia_cublas_cu12-12.8.4.1-py3-none-manylinux_2_27_x86_64.whl
(594.3 MB)
                         594.3/594.3 MB
64.0 MB/s  0:00:05m0:00:0100:01
Downloading nvidia_cuda_cupti_cu12-12.8.90-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (10.2 MB)
                         10.2/10.2 MB
81.3 MB/s  0:00:00
Downloading nvidia_cuda_nvrtc_cu12-12.8.93-py3-none-
manylinux2010_x86_64.manylinux_2_12_x86_64.whl (88.0 MB)
                         88.0/88.0 MB
105.4 MB/s  0:00:00m0:00:0100:01
Downloading nvidia_cuda_runtime_cu12-12.8.90-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (954 kB)
                         954.8/954.8 kB
21.3 MB/s  0:00:00
Downloading nvidia_cudnn_cu12-9.10.2.21-py3-none-manylinux_2_27_x86_64.whl
(706.8 MB)
                         706.8/706.8 MB
54.7 MB/s  0:00:06m0:00:0100:01
Downloading nvidia_cufft_cu12-11.3.3.83-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (193.1 MB)
                         193.1/193.1 MB
107.3 MB/s  0:00:010:00:0100:01
Downloading nvidia_cufile_cu12-1.13.1.3-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (1.2 MB)
                         1.2/1.2 MB
26.6 MB/s  0:00:00
Downloading nvidia_curand_cu12-10.3.9.90-py3-none-
manylinux_2_27_x86_64.whl (63.6 MB)
                         63.6/63.6 MB
104.7 MB/s  0:00:00m0:00:0100:01
Downloading nvidia_cusolver_cu12-11.7.3.90-py3-none-
manylinux_2_27_x86_64.whl (267.5 MB)
                         267.5/267.5 MB
107.0 MB/s  0:00:020:00:0100:01
Downloading nvidia_cusparse_cu12-12.5.8.93-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (288.2 MB)
                         288.2/288.2 MB
102.2 MB/s  0:00:020:00:0100:01
Downloading nvidia_cusparselt_cu12-0.7.1-py3-none-manylinux2014_x86_64.whl
(287.2 MB)
                         287.2/287.2 MB
103.0 MB/s  0:00:020:00:0100:01
Downloading nvidia_nccl_cu12-2.27.5-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (322.3 MB)
```

```
                         322.3/322.3 MB
41.9 MB/s  0:00:07m0:00:0100:01
Downloading nvidia_nvjitlink_cu12-12.8.93-py3-none-
manylinux2010_x86_64.manylinux_2_12_x86_64.whl (39.3 MB)
                         39.3/39.3 MB
42.3 MB/s  0:00:00m0:00:0100:01
Downloading nvidia_nvshmem_cu12-3.4.5-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (139.1 MB)
                         139.1/139.1 MB
103.9 MB/s  0:00:010:00:0100:01
Downloading nvidia_nvtx_cu12-12.8.90-py3-none-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (89 kB)
Downloading
triton-3.6.0-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (188.3
MB)
                         188.3/188.3 MB
106.6 MB/s  0:00:010:00:0100:01
Downloading cuda_pathfinder-1.4.3-py3-none-any.whl (47 kB)
Using cached datasets-4.8.2-py3-none-any.whl (526 kB)
Using cached dill-0.4.1-py3-none-any.whl (120 kB)
Downloading fsspec-2026.2.0-py3-none-any.whl (202 kB)
Using cached huggingface_hub-1.7.1-py3-none-any.whl (616 kB)
Using cached
hf_xet-1.4.2-cp37-abi3-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (4.2 MB)
Using cached multiprocess-0.70.19-py313-none-any.whl (156 kB)
Using cached transformers-5.3.0-py3-none-any.whl (10.7 MB)
Using cached
tokenizers-0.22.2-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.3
MB)
Using cached sentence_transformers-5.3.0-py3-none-any.whl (512 kB)
Downloading aiohttp-3.13.3-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (1.7 MB)
                         1.7/1.7 MB
41.0 MB/s  0:00:00
Downloading multidict-6.7.1-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (254 kB)
Downloading yarl-1.23.0-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (101 kB)
Downloading aiohappyeyeballs-2.6.1-py3-none-any.whl (15 kB)
Downloading aiosignal-1.4.0-py3-none-any.whl (7.5 kB)
Downloading filelock-3.25.2-py3-none-any.whl (26 kB)
Downloading frozenlist-1.8.0-cp313-cp313-
manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (234 kB)
Downloading networkx-3.6.1-py3-none-any.whl (2.1 MB)
                         2.1/2.1 MB
40.9 MB/s  0:00:00
Downloading propcache-0.4.1-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (204 kB)
```

```
Downloading pyarrow-23.0.1-cp313-cp313-manylinux_2_28_x86_64.whl (47.6 MB)
                           47.6/47.6 MB
102.4 MB/s  0:00:00m0:00:01
Using cached regex-2026.2.28-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (802 kB)
Using cached
safetensors-0.7.0-cp38-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (507
kB)
Downloading sympy-1.14.0-py3-none-any.whl (6.3 MB)
                           6.3/6.3 MB
71.5 MB/s  0:00:00
Downloading mpmath-1.3.0-py3-none-any.whl (536 kB)
                           536.2/536.2 kB
9.4 MB/s  0:00:00
Downloading
pandas-3.0.1-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (10.9
MB)
                           10.9/10.9 MB
86.0 MB/s  0:00:00
Downloading
scikit_learn-1.8.0-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
(8.9 MB)
                           8.9/8.9 MB
82.4 MB/s  0:00:00
Downloading joblib-1.5.3-py3-none-any.whl (309 kB)
Downloading
scipy-1.17.1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (35.2
MB)
                           35.2/35.2 MB
99.8 MB/s  0:00:006m0:00:01
Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Using cached typer-0.24.1-py3-none-any.whl (56 kB)
Using cached annotated_doc-0.0.4-py3-none-any.whl (5.3 kB)
Downloading click-8.3.1-py3-none-any.whl (108 kB)
Using cached rich-14.3.3-py3-none-any.whl (310 kB)
Using cached markdown_it_py-4.0.0-py3-none-any.whl (87 kB)
Using cached mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Using cached shellingham-1.5.4-py2.py3-none-any.whl (9.8 kB)
Using cached xxhash-3.6.0-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (193 kB)
Installing collected packages: nvidia-cusparselt-cu12, mpmath, xxhash, triton,
threadpoolctl, sympy, shellingham, safetensors, regex, pyarrow, propcache,
nvidia-nvtx-cu12, nvidia-nvshmem-cu12, nvidia-nvjitlink-cu12, nvidia-nccl-cu12,
nvidia-curand-cu12, nvidia-cufile-cu12, nvidia-cuda-runtime-cu12, nvidia-cuda-
nvrtc-cu12, nvidia-cuda-cupti-cu12, nvidia-cublas-cu12, numpy, networkx,
multidict, mdurl, joblib, hf-xet, fsspec, frozenlist, filelock, dill, cuda-
pathfinder, click, annotated-doc, aiohappyeyeballs, yarl, scipy, pandas, nvidia-
cusparse-cu12, nvidia-cufft-cu12, nvidia-cudnn-cu12, multiprocess, markdown-it-
```

```
py, cuda-bindings, aiosignal, scikit-learn, rich, nvidia-cusolver-cu12, aiohttp,
typer, torch, huggingface-hub, tokenizers, datasets, transformers, sentence-
transformers
                            56/56
[sentence-transformers]sformers]ub]cu12]2]2]
Successfully installed aiohappyeyeballs-2.6.1 aiohttp-3.13.3
aiosignal-1.4.0 annotated-doc-0.0.4 click-8.3.1 cuda-bindings-12.9.4 cuda-
pathfinder-1.4.3 datasets-4.8.2 dill-0.4.1 filelock-3.25.2 frozenlist-1.8.0
fsspec-2026.2.0 hf-xet-1.4.2 huggingface-hub-1.7.1 joblib-1.5.3 markdown-it-
py-4.0.0 mdurl-0.1.2 mpmath-1.3.0 multidict-6.7.1 multiprocess-0.70.19
networkx-3.6.1 numpy-2.4.3 nvidia-cublas-cu12-12.8.4.1 nvidia-cuda-cupti-
cu12-12.8.90 nvidia-cuda-nvrtc-cu12-12.8.93 nvidia-cuda-runtime-cu12-12.8.90
nvidia-cudnn-cu12-9.10.2.21 nvidia-cufft-cu12-11.3.3.83 nvidia-cufile-
cu12-1.13.1.3 nvidia-curand-cu12-10.3.9.90 nvidia-cusolver-cu12-11.7.3.90
nvidia-cusparse-cu12-12.5.8.93 nvidia-cusparselt-cu12-0.7.1 nvidia-nccl-
cu12-2.27.5 nvidia-nvjitlink-cu12-12.8.93 nvidia-nvshmem-cu12-3.4.5 nvidia-nvtx-
cu12-12.8.90 pandas-3.0.1 propcache-0.4.1 pyarrow-23.0.1 regex-2026.2.28
rich-14.3.3 safetensors-0.7.0 scikit-learn-1.8.0 scipy-1.17.1 sentence-
transformers-5.3.0 shellingham-1.5.4 sympy-1.14.0 threadpoolctl-3.6.0
tokenizers-0.22.2 torch-2.10.0 transformers-5.3.0 triton-3.6.0 typer-0.24.1
xxhash-3.6.0 yarl-1.23.0
```

[3]:
```python
import os
import re
import json
import math
import time
import random
import shutil
from dataclasses import dataclass
from typing import Dict, List, Tuple, Optional

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
from sentence_transformers import SentenceTransformer


# ============================================================
# Config
# ============================================================

@dataclass
```

```python
class Config:
    # Dataset
    train_subset: int = 300
    test_subset: int = 100

    # Models
    llm_name: str = "Qwen/Qwen2.5-1.5B-Instruct"
    # llm_name: str = "Qwen/Qwen2.5-3B-Instruct"
    embed_name: str = "sentence-transformers/all-MiniLM-L6-v2"

    # Oracle search: denser around strong region
    oracle_budgets: Tuple[int, ...] = (
        96, 104, 112, 120, 128, 136, 144, 152, 160, 176, 192, 208, 220
    )

    # Supervised warm-start
    warmstart_epochs: int = 15
    warmstart_learning_rate: float = 5e-4

    # RL fine-tuning
    rl_epochs: int = 5
    policy_learning_rate: float = 3e-4
    value_learning_rate: float = 8e-4
    entropy_coef: float = 0.002
    value_loss_coef: float = 0.5
    grad_clip_norm: float = 1.0

    # Reward
    reward_token_penalty: float = 0.00005
    use_soft_reward: bool = False
    soft_reward_floor: float = 0.0
    exact_match_bonus: float = 0.25

    # Thinking budget range
    min_budget: int = 64
    max_budget: int = 220

    # Final answer generation budget
    answer_max_new_tokens: int = 20

    # Generation
    do_sample: bool = False
    temperature: float = 0.0

    # RL exploration warmup
    warmup_epochs_rl: int = 2
    warmup_mix_rl: float = 0.25
```

```
    warmup_min_budget_high: int = 128

    # Hardware
    seed: int = 42
    device: str = "cuda" if torch.cuda.is_available() else "cpu"

    # Logging
    print_every: int = 20
    debug_examples: int = 3
    eval_print_every: int = 10

    # Cache / outputs
    cache_dir: str = "./cache_rl_two_stage_v9"
    clear_cache_on_start: bool = True
    oracle_cache_path: str = "oracle_budget_labels_v9.json"

    # Saved models
    best_warmstart_policy_path: str = "best_warmstart_policy_v9.pt"
    best_warmstart_policy_by_acc_path: str = "best_warmstart_policy_by_acc_v9.
 ↪pt"
    best_policy_path: str = "best_policy_v9.pt"
    best_policy_by_acc_path: str = "best_policy_by_acc_v9.pt"
    best_value_path: str = "best_value_v9.pt"
    final_policy_path: str = "two_stage_continuous_budget_policy_v9_final.pt"
    final_value_path: str = "two_stage_value_net_v9_final.pt"

    # Baselines
    baseline_budgets: Tuple[int, ...] = (96, 128, 144, 160, 176, 192, 220)


CFG = Config()


# ============================================================
# Utilities
# ============================================================

def set_seed(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)


def ensure_dir(path: str) -> None:
    os.makedirs(path, exist_ok=True)
```

```python
def reset_cache_dir(cache_dir: str, retries: int = 5, delay: float = 0.5) ->␣
 ↪None:
    """
    More robust cache reset for environments where directory handles may linger.
    """
    if not os.path.exists(cache_dir):
        os.makedirs(cache_dir, exist_ok=True)
        return

    last_error = None

    for attempt in range(retries):
        try:
            if os.path.exists(cache_dir):
                # First try normal removal
                shutil.rmtree(cache_dir, ignore_errors=False)

            if os.path.exists(cache_dir):
                # Fallback: walk and remove contents manually
                for root, dirs, files in os.walk(cache_dir, topdown=False):
                    for file_name in files:
                        file_path = os.path.join(root, file_name)
                        try:
                            os.remove(file_path)
                        except FileNotFoundError:
                            pass
                    for dir_name in dirs:
                        dir_path = os.path.join(root, dir_name)
                        try:
                            os.rmdir(dir_path)
                        except OSError:
                            pass
                if os.path.exists(cache_dir):
                    os.rmdir(cache_dir)

            break
        except OSError as e:
            last_error = e
            time.sleep(delay)

    if os.path.exists(cache_dir):
        raise RuntimeError(
            f"Failed to fully clear cache directory after retries: {cache_dir}"
        ) from last_error
```

```
    os.makedirs(cache_dir, exist_ok=True)


def clamp_budget(x: float, min_budget: int, max_budget: int) -> int:
    return int(max(min_budget, min(max_budget, round(x))))


def denormalize_budget(x: float, min_budget: int, max_budget: int) -> float:
    return x * (max_budget - min_budget) + min_budget


def budget_to_normalized(budget: int, min_budget: int, max_budget: int) ->␣
 ↪float:
    return float(budget - min_budget) / float(max_budget - min_budget)


def normalize_scalar_list(values: List[float]) -> List[float]:
    if len(values) == 0:
        return values
    arr = np.array(values, dtype=np.float32)
    mean = float(arr.mean())
    std = float(arr.std()) + 1e-8
    return ((arr - mean) / std).tolist()


def clean_model_output(text: str) -> str:
    if text is None:
        return ""

    cleaned = text

    stop_markers = [
        "\nHuman:",
        "\nAssistant:",
        "\nUser:",
        "\nSystem:",
        "Human:",
        "Assistant:",
        "User:",
        "System:",
        "You are an AI assistant",
        "Provide a detailed answer",
        "I'll respond in English",
        "<|im_end|>",
        "<|endoftext|>",
    ]
```

```
    for marker in stop_markers:
        if marker in cleaned:
            cleaned = cleaned.split(marker)[0]

    return cleaned.strip()


# ============================================================
# GSM8K helpers
# ============================================================

def extract_gsm8k_final_answer(answer_text: str) -> str:
    if "####" in answer_text:
        return answer_text.split("####")[-1].strip()
    return answer_text.strip()


def normalize_numeric_string(text: str) -> str:
    if text is None:
        return ""

    text = text.strip()
    text = text.replace(",", "")
    text = text.replace("$", "")
    text = text.replace("%", "")
    text = text.strip()

    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if not matches:
        return text.lower().strip()
    return matches[-1]


def parse_final_answer(output_text: str) -> str:
    text = clean_model_output(output_text)

    patterns = [
        r"FINAL ANSWER\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Final Answer\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Answer\s*:\s*(-?\d+(?:\.\d+)?)",
    ]

    for pattern in patterns:
        m = re.search(pattern, text, re.IGNORECASE)
        if m:
            return normalize_numeric_string(m.group(1))
```

```
    lines = [line.strip() for line in text.splitlines() if line.strip()]
    first_line = lines[0] if lines else text.strip()

    m = re.search(r"-?\d+(?:\.\d+)?", first_line)
    if m:
        return normalize_numeric_string(m.group(0))

    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if matches:
        return normalize_numeric_string(matches[0])

    return normalize_numeric_string(text)


def is_correct_prediction(pred: str, gold: str) -> int:
    return int(pred == gold)


def soft_numeric_match(pred: str, gold: str, floor: float = 0.0) -> float:
    try:
        p = float(pred)
        g = float(gold)
        error = abs(p - g)
        denom = max(abs(g), 1.0)
        score = 1.0 - (error / denom)
        return float(max(floor, min(1.0, score)))
    except Exception:
        return 0.0


def compute_reward(
    pred: str,
    gold: str,
    thinking_tokens_used: int,
    cfg: Config
) -> Tuple[float, float]:
    exact = float(is_correct_prediction(pred, gold))

    if cfg.use_soft_reward:
        answer_score = max(exact, soft_numeric_match(pred, gold, floor=cfg.
 ↪soft_reward_floor))
    else:
        answer_score = exact

    reward = answer_score - cfg.reward_token_penalty *␣
 ↪float(thinking_tokens_used)
```

```
    if exact > 0.5:
        reward += cfg.exact_match_bonus

    return reward, exact


# ============================================================
# Prompts
# ============================================================

def build_thinking_prompt(question: str) -> str:
    return (
        "Solve the following grade-school math problem carefully.\n"
        "Reason step by step.\n"
        "Keep the reasoning concise and relevant.\n"
        "Do not start a conversation.\n"
        "Do not add extra instructions.\n"
        "Only produce reasoning.\n\n"
        f"Question: {question}\n\n"
        "Reasoning:"
    )


def build_answer_prompt(question: str, thinking_text: str) -> str:
    return (
        "Use the reasoning below to produce the final numeric answer.\n"
        "Return only one line in this exact format:\n"
        "FINAL ANSWER: <number>\n\n"
        f"Question: {question}\n\n"
        f"Reasoning:\n{thinking_text}\n\n"
        "FINAL ANSWER:"
    )


# ============================================================
# Featurizer
# ============================================================

class QuestionFeaturizer:
    def __init__(self, embed_model_name: str, device: str):
        self.embedder = SentenceTransformer(embed_model_name, device=device)

    def handcrafted_features(self, question: str) -> np.ndarray:
        q = question.lower()
        tokens = q.split()

        length_feat = len(tokens)
```

```
        num_count = len(re.findall(r"\d+", q))
        sentence_len_chars = len(q)
        sentence_count = max(1, len(re.findall(r"[.!?]", q)))

        keywords = [
            "total", "left", "remaining", "each", "every",
            "more", "less", "after", "before", "twice",
            "half", "times", "altogether", "difference",
            "sum", "product", "cost", "price", "sold",
            "per", "average", "equal", "share", "then",
            "gave", "bought", "earned", "minutes", "hours",
            "days", "weeks", "fraction", "ratio", "percent",
            "another", "still", "now", "together"
        ]
        keyword_hits = [1.0 if kw in q else 0.0 for kw in keywords]

        plus_cue = 1.0 if any(w in q for w in ["more", "sum", "altogether",␣
↪"total", "together"]) else 0.0
        minus_cue = 1.0 if any(w in q for w in ["left", "remaining",␣
↪"difference", "less", "still"]) else 0.0
        mult_cue = 1.0 if any(w in q for w in ["each", "every", "times",␣
↪"product", "twice"]) else 0.0
        div_cue = 1.0 if any(w in q for w in ["per", "average", "equally",␣
↪"half", "share", "ratio"]) else 0.0
        multi_step_cue = 1.0 if any(w in q for w in ["then", "after", "before",␣
↪"altogether", "remaining", "now"]) else 0.0

        unit_time_cue = 1.0 if any(w in q for w in ["minutes", "hours", "days",␣
↪"weeks"]) else 0.0
        money_cue = 1.0 if "$" in q or any(w in q for w in ["dollar",␣
↪"dollars", "cost", "price"]) else 0.0
        fraction_cue = 1.0 if any(w in q for w in ["half", "fraction",␣
↪"percent", "ratio"]) else 0.0

        feats = np.array(
            [
                length_feat,
                num_count,
                sentence_len_chars,
                sentence_count,
                plus_cue,
                minus_cue,
                mult_cue,
                div_cue,
                multi_step_cue,
                unit_time_cue,
```

```
                money_cue,
                fraction_cue,
            ] + keyword_hits,
            dtype=np.float32
        )
        return feats

    def encode(self, question: str) -> np.ndarray:
        emb = self.embedder.encode(
            question,
            convert_to_numpy=True,
            normalize_embeddings=True
        ).astype(np.float32)
        hand = self.handcrafted_features(question)
        return np.concatenate([emb, hand], axis=0)


# ============================================================
# Policy + Value Networks
# ============================================================

class ContinuousBudgetPolicy(nn.Module):
    def __init__(self, input_dim: int, hidden_dim: int = 256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
        )
        self.mean_head = nn.Linear(hidden_dim, 1)
        self.log_std = nn.Parameter(torch.tensor([0.0], dtype=torch.float32))

    def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        h = self.net(x)
        mean = self.mean_head(h)
        log_std = self.log_std.expand_as(mean)
        return mean, log_std

    def sample_budget(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.
↪Tensor, torch.Tensor]:
        mean, log_std = self.forward(x)
        std = torch.exp(log_std)
        dist = torch.distributions.Normal(mean, std)
        raw = dist.rsample()
        normalized = torch.sigmoid(raw)
        log_prob = dist.log_prob(raw).sum(dim=-1)
```

```
        entropy = dist.entropy().sum(dim=-1)
        return normalized, log_prob, entropy

    def deterministic_budget(self, x: torch.Tensor) -> torch.Tensor:
        mean, _ = self.forward(x)
        return torch.sigmoid(mean)


class ValueNetwork(nn.Module):
    def __init__(self, input_dim: int, hidden_dim: int = 256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x).squeeze(-1)


# ============================================================
# LLM wrapper
# ============================================================

class LLMReasoner:
    def __init__(self, model_name: str, device: str):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)

        if self.tokenizer.pad_token is None:
            self.tokenizer.pad_token = self.tokenizer.eos_token

        dtype = torch.float16 if device == "cuda" else torch.float32

        self.model = AutoModelForCausalLM.from_pretrained(
            model_name,
            dtype=dtype,
            device_map="auto" if device == "cuda" else None
        )
        self.model.eval()
        self.device = device

        if hasattr(self.model, "generation_config") and self.model.
 ↪generation_config is not None:
            self.model.generation_config.do_sample = False
```

```
            self.model.generation_config.temperature = None
            self.model.generation_config.top_p = None
            self.model.generation_config.top_k = None

    def generate(
        self,
        prompt: str,
        max_new_tokens: int,
        do_sample: bool = False,
        temperature: float = 0.0
    ) -> Dict:
        inputs = self.tokenizer(prompt, return_tensors="pt").to(self.model.
 ↪device)
        input_len = inputs["input_ids"].shape[1]

        gen_kwargs = {
            "max_new_tokens": max_new_tokens,
            "do_sample": do_sample,
            "pad_token_id": self.tokenizer.eos_token_id,
            "eos_token_id": self.tokenizer.eos_token_id,
        }

        if do_sample:
            gen_kwargs["temperature"] = temperature

        with torch.no_grad():
            outputs = self.model.generate(**inputs, **gen_kwargs)

        gen_ids = outputs[0][input_len:]
        text = self.tokenizer.decode(gen_ids, skip_special_tokens=True)
        text = clean_model_output(text)
        tokens_used = len(gen_ids)

        return {"text": text, "tokens_used": tokens_used}


# ============================================================
# Caching
# ============================================================

def cache_key(stage: str, question: str, extra: str, model_name: str) -> str:
    import hashlib
    raw = f"{stage}|||{model_name}|||{question}|||{extra}"
    return hashlib.md5(raw.encode("utf-8")).hexdigest()


def load_from_cache(cache_dir: str, key: str) -> Optional[Dict]:
```

```
    path = os.path.join(cache_dir, f"{key}.json")
    if os.path.exists(path):
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    return None


def save_to_cache(cache_dir: str, key: str, value: Dict) -> None:
    path = os.path.join(cache_dir, f"{key}.json")
    with open(path, "w", encoding="utf-8") as f:
        json.dump(value, f, ensure_ascii=False, indent=2)


# ============================================================
# Two-stage example run
# ============================================================

def run_two_stage_example(
    question: str,
    gold_answer: str,
    thinking_budget: int,
    llm: LLMReasoner,
    cfg: Config,
) -> Dict:
    ensure_dir(cfg.cache_dir)

    think_prompt = build_thinking_prompt(question)
    think_key = cache_key("thinking", question, f"budget={thinking_budget}",␣
 ↪cfg.llm_name)

    cached_think = load_from_cache(cfg.cache_dir, think_key)
    if cached_think is not None:
        thinking_text = clean_model_output(cached_think["text"])
        thinking_tokens_used = cached_think["tokens_used"]
    else:
        think_result = llm.generate(
            prompt=think_prompt,
            max_new_tokens=thinking_budget,
            do_sample=cfg.do_sample,
            temperature=cfg.temperature
        )
        thinking_text = clean_model_output(think_result["text"])
        thinking_tokens_used = think_result["tokens_used"]
        save_to_cache(cfg.cache_dir, think_key, {"text": thinking_text,␣
 ↪"tokens_used": thinking_tokens_used})

    answer_prompt = build_answer_prompt(question, thinking_text)
```

```
    answer_key = cache_key("answer", question, thinking_text, cfg.llm_name)

    cached_answer = load_from_cache(cfg.cache_dir, answer_key)
    if cached_answer is not None:
        answer_text = clean_model_output(cached_answer["text"])
        answer_tokens_used = cached_answer["tokens_used"]
    else:
        answer_result = llm.generate(
            prompt=answer_prompt,
            max_new_tokens=cfg.answer_max_new_tokens,
            do_sample=False,
            temperature=0.0
        )
        answer_text = clean_model_output(answer_result["text"])
        answer_tokens_used = answer_result["tokens_used"]
        save_to_cache(cfg.cache_dir, answer_key, {"text": answer_text,␣
 ↪"tokens_used": answer_tokens_used})

    pred = parse_final_answer(answer_text)
    reward, exact = compute_reward(pred, gold_answer, thinking_tokens_used, cfg)

    return {
        "prediction": pred,
        "gold": gold_answer,
        "correct": int(exact),
        "thinking_tokens_used": thinking_tokens_used,
        "answer_tokens_used": answer_tokens_used,
        "reward": reward,
        "thinking_text": thinking_text,
        "answer_text": answer_text,
        "thinking_budget": thinking_budget,
    }


# ============================================================
# Debug
# ============================================================

def print_debug_examples(
    data: List[Dict],
    llm: LLMReasoner,
    cfg: Config,
    budget: int,
    n: int = 3
) -> None:
    print("\n" + "=" * 100)
    print(f"DEBUG EXAMPLES WITH FIXED THINKING BUDGET = {budget}")
```

```
    print("=" * 100)

    for i, item in enumerate(data[:n]):
        question = item["question"]
        gold =␣
 ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        result = run_two_stage_example(question, gold, budget, llm, cfg)

        print(f"\n[Example {i+1}]")
        print("-" * 100)
        print("QUESTION:")
        print(question)
        print("\nTHINKING TEXT:")
        print(result["thinking_text"])
        print("\nANSWER TEXT:")
        print(result["answer_text"])
        print("\nPARSED PREDICTION:", result["prediction"])
        print("GOLD ANSWER      :", gold)
        print("CORRECT          :", result["correct"])
        print("THINK TOKENS     :", result["thinking_tokens_used"])
        print("REWARD           :", result["reward"])
        print("-" * 100)


# ============================================================
# Baselines and evaluation
# ============================================================

def evaluate_fixed_budget(
    data: List[Dict],
    llm: LLMReasoner,
    cfg: Config,
    fixed_budget: int,
) -> Dict:
    corrects = []
    thinking_tokens = []
    rewards = []

    print(f"\nEvaluating fixed budget = {fixed_budget} on {len(data)} examples..
 ↪.")

    for idx, item in enumerate(data):
        question = item["question"]
        gold =␣
 ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        result = run_two_stage_example(question, gold, fixed_budget, llm, cfg)
```

```
        corrects.append(result["correct"])
        thinking_tokens.append(result["thinking_tokens_used"])
        rewards.append(result["reward"])

        if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == len(data):
            print(
                f"  Fixed budget {fixed_budget} | Done {idx+1}/{len(data)} | "
                f"Acc={np.mean(corrects):.4f} | AvgThinkTokens={np.
 ↪mean(thinking_tokens):.2f} | "
                f"AvgReward={np.mean(rewards):.4f}"
            )

    return {
        "accuracy": float(np.mean(corrects)) if corrects else 0.0,
        "avg_thinking_tokens": float(np.mean(thinking_tokens)) if␣
 ↪thinking_tokens else 0.0,
        "avg_reward": float(np.mean(rewards)) if rewards else 0.0,
        "budget": fixed_budget,
    }


def evaluate_policy_deterministic(
    data: List[Dict],
    featurizer: QuestionFeaturizer,
    policy: ContinuousBudgetPolicy,
    llm: LLMReasoner,
    cfg: Config,
) -> Dict:
    policy.eval()

    corrects = []
    thinking_tokens = []
    rewards = []
    budgets = []

    with torch.no_grad():
        for idx, item in enumerate(data):
            question = item["question"]
            gold =␣
 ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
            feat = featurizer.encode(question)
            x = torch.tensor(feat, dtype=torch.float32, device=cfg.device).
 ↪unsqueeze(0)

            norm_budget = policy.deterministic_budget(x).item()
            budget = clamp_budget(
                denormalize_budget(norm_budget, cfg.min_budget, cfg.max_budget),
```

```
                cfg.min_budget,
                cfg.max_budget,
            )

            result = run_two_stage_example(question, gold, budget, llm, cfg)
            corrects.append(result["correct"])
            thinking_tokens.append(result["thinking_tokens_used"])
            rewards.append(result["reward"])
            budgets.append(budget)

            if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == len(data):
                print(
                    f"  Deterministic policy eval | Done {idx+1}/{len(data)} | "
                    f"Acc={np.mean(corrects):.4f} | AvgBudget={np.mean(budgets):
 ↪.2f}"
                )

    return {
        "accuracy": float(np.mean(corrects)) if corrects else 0.0,
        "avg_thinking_tokens": float(np.mean(thinking_tokens)) if␣
 ↪thinking_tokens else 0.0,
        "avg_reward": float(np.mean(rewards)) if rewards else 0.0,
        "avg_budget": float(np.mean(budgets)) if budgets else 0.0,
        "min_budget": float(np.min(budgets)) if budgets else 0.0,
        "max_budget": float(np.max(budgets)) if budgets else 0.0,
        "std_budget": float(np.std(budgets)) if budgets else 0.0,
    }


def evaluate_policy_sampled(
    data: List[Dict],
    featurizer: QuestionFeaturizer,
    policy: ContinuousBudgetPolicy,
    llm: LLMReasoner,
    cfg: Config,
) -> Dict:
    policy.eval()

    corrects = []
    thinking_tokens = []
    rewards = []
    budgets = []

    with torch.no_grad():
        for idx, item in enumerate(data):
            question = item["question"]
```

```
            gold = normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
            feat = featurizer.encode(question)
            x = torch.tensor(feat, dtype=torch.float32, device=cfg.device).unsqueeze(0)

            norm_budget, _, _ = policy.sample_budget(x)
            budget = clamp_budget(
                denormalize_budget(norm_budget.item(), cfg.min_budget, cfg.max_budget),
                cfg.min_budget,
                cfg.max_budget,
            )

            result = run_two_stage_example(question, gold, budget, llm, cfg)
            corrects.append(result["correct"])
            thinking_tokens.append(result["thinking_tokens_used"])
            rewards.append(result["reward"])
            budgets.append(budget)

            if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == len(data):
                print(
                    f"  Sampled policy eval | Done {idx+1}/{len(data)} | "
                    f"Acc={np.mean(corrects):.4f} | AvgBudget={np.mean(budgets):.2f}"
                )

    return {
        "accuracy": float(np.mean(corrects)) if corrects else 0.0,
        "avg_thinking_tokens": float(np.mean(thinking_tokens)) if thinking_tokens else 0.0,
        "avg_reward": float(np.mean(rewards)) if rewards else 0.0,
        "avg_budget": float(np.mean(budgets)) if budgets else 0.0,
        "min_budget": float(np.min(budgets)) if budgets else 0.0,
        "max_budget": float(np.max(budgets)) if budgets else 0.0,
        "std_budget": float(np.std(budgets)) if budgets else 0.0,
    }


# ============================================================
# Oracle budget construction
# ============================================================

def build_oracle_budget_labels(
    train_data: List[Dict],
    llm: LLMReasoner,
    cfg: Config,
```

```
) -> List[Dict]:
    if os.path.exists(cfg.oracle_cache_path):
        print(f"\nLoading oracle labels from {cfg.oracle_cache_path} ...")
        with open(cfg.oracle_cache_path, "r", encoding="utf-8") as f:
            return json.load(f)

    print("\nBuilding oracle budget labels...")

    oracle_records = []

    for idx, item in enumerate(train_data):
        question = item["question"]
        gold =␣
↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))

        best_budget = None
        best_reward = -1e9
        budget_rewards = {}

        for budget in cfg.oracle_budgets:
            result = run_two_stage_example(question, gold, budget, llm, cfg)
            reward = float(result["reward"])
            budget_rewards[str(budget)] = {
                "reward": reward,
                "correct": int(result["correct"]),
                "thinking_tokens_used": int(result["thinking_tokens_used"]),
                "prediction": result["prediction"],
            }

            if reward > best_reward:
                best_reward = reward
                best_budget = budget

        oracle_records.append(
            {
                "question": question,
                "gold_answer": gold,
                "best_budget": int(best_budget),
                "best_reward": float(best_reward),
                "budget_rewards": budget_rewards,
            }
        )

        if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) ==␣
↪len(train_data):
            print(
                f"  Oracle labels | Done {idx+1}/{len(train_data)} | "
```

```
                f"AvgBestBudget={np.mean([r['best_budget'] for r in␣
 ↪oracle_records]):.2f} | "
                f"AvgBestReward={np.mean([r['best_reward'] for r in␣
 ↪oracle_records]):.4f}"
            )

    with open(cfg.oracle_cache_path, "w", encoding="utf-8") as f:
        json.dump(oracle_records, f, ensure_ascii=False, indent=2)

    print(f"Saved oracle labels to {cfg.oracle_cache_path}")
    return oracle_records


# ============================================================
# Supervised warm-start
# ============================================================

def warmstart_policy_from_oracle(
    oracle_records: List[Dict],
    test_data: List[Dict],
    featurizer: QuestionFeaturizer,
    policy: ContinuousBudgetPolicy,
    llm: LLMReasoner,
    cfg: Config,
) -> None:
    print("\nStarting supervised warm-start from oracle budgets...")

    optimizer = optim.Adam(policy.parameters(), lr=cfg.warmstart_learning_rate)
    mse_loss = nn.MSELoss()

    best_eval_reward = -1e9
    best_eval_accuracy = -1.0

    for epoch in range(cfg.warmstart_epochs):
        policy.train()
        random.shuffle(oracle_records)

        losses = []
        pred_budgets = []
        target_budgets = []

        for step_idx, record in enumerate(oracle_records):
            question = record["question"]
            target_budget = int(record["best_budget"])

            feat = featurizer.encode(question)
```

```
            x = torch.tensor(feat, dtype=torch.float32, device=cfg.device).
↪unsqueeze(0)

            pred_norm = policy.deterministic_budget(x)
            target_norm = torch.tensor(
                [[budget_to_normalized(target_budget, cfg.min_budget, cfg.
↪max_budget)]],
                dtype=torch.float32,
                device=cfg.device,
            )

            sample_weight = max(0.1, float(record["best_reward"]))
            loss = sample_weight * mse_loss(pred_norm, target_norm)

            optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(policy.parameters(), cfg.
↪grad_clip_norm)
            optimizer.step()

            losses.append(float(loss.item()))

            pred_budget = clamp_budget(
                denormalize_budget(pred_norm.item(), cfg.min_budget, cfg.
↪max_budget),
                cfg.min_budget,
                cfg.max_budget,
            )
            pred_budgets.append(pred_budget)
            target_budgets.append(target_budget)

            if (step_idx + 1) % cfg.print_every == 0 or (step_idx + 1) ==␣
↪len(oracle_records):
                mae = np.mean(np.abs(np.array(pred_budgets) - np.
↪array(target_budgets)))
                print(
                    f"[Warmstart {epoch+1}/{cfg.warmstart_epochs}] Step␣
↪{step_idx+1}/{len(oracle_records)} | "
                    f"Loss={np.mean(losses):.6f} | PredAvgBudget={np.
↪mean(pred_budgets):.2f} | "
                    f"TargetAvgBudget={np.mean(target_budgets):.2f} | MAE={mae:.
↪2f}"
                )

        print(
            f"Warmstart epoch {epoch+1} summary | "
```

```
            f"Loss={np.mean(losses):.6f} | PredAvgBudget={np.mean(pred_budgets):
 ↪.2f} | "
            f"TargetAvgBudget={np.mean(target_budgets):.2f}"
        )

        print("\nWarm-start deterministic evaluation on test set...")
        eval_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
 ↪llm, cfg)

        if eval_det["avg_reward"] > best_eval_reward:
            best_eval_reward = eval_det["avg_reward"]
            torch.save(policy.state_dict(), cfg.best_warmstart_policy_path)
            print(f"Saved new best warm-start policy by reward =␣
 ↪{best_eval_reward:.4f}\n")

        if eval_det["accuracy"] > best_eval_accuracy:
            best_eval_accuracy = eval_det["accuracy"]
            torch.save(policy.state_dict(), cfg.
 ↪best_warmstart_policy_by_acc_path)
            print(f"Saved new best warm-start policy by accuracy =␣
 ↪{best_eval_accuracy:.4f}\n")


# ============================================================
# RL helper
# ============================================================

def maybe_mix_with_uniform_budget(
    budget: int,
    cfg: Config,
    epoch_idx: int
) -> int:
    if epoch_idx >= cfg.warmup_epochs_rl:
        return budget

    if random.random() < cfg.warmup_mix_rl:
        return random.randint(cfg.warmup_min_budget_high, cfg.max_budget)

    return budget


# ============================================================
# RL fine-tuning
# ============================================================

def train_policy_with_value(
    train_data: List[Dict],
```

```
    test_data: List[Dict],
    featurizer: QuestionFeaturizer,
    policy: ContinuousBudgetPolicy,
    value_net: ValueNetwork,
    llm: LLMReasoner,
    cfg: Config,
) -> None:
    print("\nStarting RL fine-tuning...")

    policy_optimizer = optim.Adam(policy.parameters(), lr=cfg.
↪policy_learning_rate)
    value_optimizer = optim.Adam(value_net.parameters(), lr=cfg.
↪value_learning_rate)

    best_test_reward = -1e9
    best_test_accuracy = -1.0

    for epoch in range(cfg.rl_epochs):
        policy.train()
        value_net.train()
        random.shuffle(train_data)

        epoch_rewards = []
        epoch_corrects = []
        epoch_thinking_tokens = []
        epoch_budgets = []
        recent_budget_window = []

        for step_idx, item in enumerate(train_data):
            question = item["question"]
            gold =␣
↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))

            feat = featurizer.encode(question)
            x = torch.tensor(feat, dtype=torch.float32, device=cfg.device).
↪unsqueeze(0)

            norm_budget, log_prob, entropy = policy.sample_budget(x)
            sampled_budget = clamp_budget(
                denormalize_budget(norm_budget.item(), cfg.min_budget, cfg.
↪max_budget),
                cfg.min_budget,
                cfg.max_budget,
            )

            budget = maybe_mix_with_uniform_budget(sampled_budget, cfg, epoch)
```

```
            result = run_two_stage_example(question, gold, budget, llm, cfg)
            reward = float(result["reward"])
            reward_tensor = torch.tensor([reward], dtype=torch.float32,␣
↪device=cfg.device)

            value_pred = value_net(x)
            advantage = reward_tensor - value_pred.detach()

            policy_loss = -(log_prob * advantage + cfg.entropy_coef * entropy).
↪mean()
            value_loss = nn.functional.mse_loss(value_pred, reward_tensor)

            policy_optimizer.zero_grad()
            policy_loss.backward()
            torch.nn.utils.clip_grad_norm_(policy.parameters(), cfg.
↪grad_clip_norm)
            policy_optimizer.step()

            value_optimizer.zero_grad()
            (cfg.value_loss_coef * value_loss).backward()
            torch.nn.utils.clip_grad_norm_(value_net.parameters(), cfg.
↪grad_clip_norm)
            value_optimizer.step()

            epoch_rewards.append(reward)
            epoch_corrects.append(result["correct"])
            epoch_thinking_tokens.append(result["thinking_tokens_used"])
            epoch_budgets.append(budget)

            recent_budget_window.append(budget)
            if len(recent_budget_window) > 10:
                recent_budget_window.pop(0)

            if (step_idx + 1) % cfg.print_every == 0 or (step_idx + 1) ==␣
↪len(train_data):
                rewards_norm = normalize_scalar_list(epoch_rewards)
                print(
                    f"[RL {epoch+1}/{cfg.rl_epochs}] Step {step_idx+1}/
↪{len(train_data)} | "
                    f"AvgReward={np.mean(epoch_rewards):.4f} | "
                    f"NormRewardMean={np.mean(rewards_norm):.4f} | "
                    f"Acc={np.mean(epoch_corrects):.4f} | "
                    f"AvgThinkTokens={np.mean(epoch_thinking_tokens):.2f} | "
                    f"AvgBudget={np.mean(epoch_budgets):.2f} | "
                    f"MinBudget={np.min(epoch_budgets):.2f} | "
                    f"MaxBudget={np.max(epoch_budgets):.2f} | "
```

```
                    f"StdBudget={np.std(epoch_budgets):.2f}"
                )
                print(f"Recent budgets: {recent_budget_window}")

        print("\n" + "=" * 90)
        print(f"RL Epoch {epoch+1} summary")
        print(f"Train Accuracy        : {np.mean(epoch_corrects):.4f}")
        print(f"Train Avg ThinkTokens : {np.mean(epoch_thinking_tokens):.2f}")
        print(f"Train Avg Reward      : {np.mean(epoch_rewards):.4f}")
        print(f"Train Avg Budget      : {np.mean(epoch_budgets):.2f}")
        print(f"Train Min Budget      : {np.min(epoch_budgets):.2f}")
        print(f"Train Max Budget      : {np.max(epoch_budgets):.2f}")
        print(f"Train Std Budget      : {np.std(epoch_budgets):.2f}")

        print("\nDeterministic policy evaluation on test set...")
        eval_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
 ↪llm, cfg)
        print(f"Test Accuracy         : {eval_det['accuracy']:.4f}")
        print(f"Test Avg ThinkTokens  : {eval_det['avg_thinking_tokens']:.2f}")
        print(f"Test Avg Reward       : {eval_det['avg_reward']:.4f}")
        print(f"Test Avg Budget       : {eval_det['avg_budget']:.2f}")
        print(f"Test Min Budget       : {eval_det['min_budget']:.2f}")
        print(f"Test Max Budget       : {eval_det['max_budget']:.2f}")
        print(f"Test Std Budget       : {eval_det['std_budget']:.2f}")
        print("=" * 90 + "\n")

        if eval_det["avg_reward"] > best_test_reward:
            best_test_reward = eval_det["avg_reward"]
            torch.save(policy.state_dict(), cfg.best_policy_path)
            torch.save(value_net.state_dict(), cfg.best_value_path)
            print(f"Saved new best RL models by reward = {best_test_reward:.
 ↪4f}\n")

        if eval_det["accuracy"] > best_test_accuracy:
            best_test_accuracy = eval_det["accuracy"]
            torch.save(policy.state_dict(), cfg.best_policy_by_acc_path)
            print(f"Saved new best RL policy by accuracy = {best_test_accuracy:.
 ↪4f}\n")


# ================================================================
# Main
# ================================================================

def main() -> None:
    set_seed(CFG.seed)
```

```
    print("torch version:", torch.__version__)
    print("cuda available:", torch.cuda.is_available())
    print("cuda device count:", torch.cuda.device_count())
    if torch.cuda.is_available():
        print("gpu name:", torch.cuda.get_device_name(0))
    else:
        print("running on cpu")

    if CFG.clear_cache_on_start:
        print(f"\nClearing old cache at: {CFG.cache_dir}")
        reset_cache_dir(CFG.cache_dir)
    else:
        ensure_dir(CFG.cache_dir)

    print(f"Using device: {CFG.device}")
    print("Loading GSM8K...")
    ds = load_dataset("gsm8k", "main")

    train_data = list(ds["train"].select(range(min(CFG.train_subset,␣
↪len(ds["train"])))))
    test_data = list(ds["test"].select(range(min(CFG.test_subset,␣
↪len(ds["test"])))))

    print(f"Train subset: {len(train_data)}")
    print(f"Test subset : {len(test_data)}")

    print("Loading featurizer...")
    featurizer = QuestionFeaturizer(CFG.embed_name, CFG.device)
    sample_feat = featurizer.encode(train_data[0]["question"])
    input_dim = sample_feat.shape[0]

    print("Loading policy and value network...")
    policy = ContinuousBudgetPolicy(input_dim=input_dim).to(CFG.device)
    value_net = ValueNetwork(input_dim=input_dim).to(CFG.device)

    # Bias upward so early budgets are not too low
    with torch.no_grad():
        policy.mean_head.bias.fill_(0.45)

    print("Loading LLM...")
    llm = LLMReasoner(CFG.llm_name, CFG.device)

    print_debug_examples(train_data, llm, CFG, budget=160, n=CFG.debug_examples)

    print("\nRunning fixed-budget baselines before warm-start/RL on TEST set...
↪\n")
    for fixed_budget in CFG.baseline_budgets:
```

```
        res = evaluate_fixed_budget(test_data, llm, CFG, fixed_budget)
        print(
            f"\nFinal fixed budget {fixed_budget:>3} | "
            f"Acc={res['accuracy']:.4f} | "
            f"AvgThinkTokens={res['avg_thinking_tokens']:.2f} | "
            f"AvgReward={res['avg_reward']:.4f}"
        )

    oracle_records = build_oracle_budget_labels(train_data, llm, CFG)

    warmstart_policy_from_oracle(
        oracle_records=oracle_records,
        test_data=test_data,
        featurizer=featurizer,
        policy=policy,
        llm=llm,
        cfg=CFG,
    )

    # Prefer best-by-accuracy if available
    if os.path.exists(CFG.best_warmstart_policy_by_acc_path):
        print("Loading best warm-start policy by accuracy...")
        policy.load_state_dict(torch.load(CFG.
↪best_warmstart_policy_by_acc_path, map_location=CFG.device))
    elif os.path.exists(CFG.best_warmstart_policy_path):
        print("Loading best warm-start policy by reward...")
        policy.load_state_dict(torch.load(CFG.best_warmstart_policy_path,␣
↪map_location=CFG.device))

    print("\nEvaluating deterministic policy right after warm-start...")
    warm_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
↪llm, CFG)
    print(json.dumps({"warmstart_deterministic_eval": warm_det}, indent=2))

    print("\nEvaluating sampled policy right after warm-start...")
    warm_samp = evaluate_policy_sampled(test_data, featurizer, policy, llm, CFG)
    print(json.dumps({"warmstart_sampled_eval": warm_samp}, indent=2))

    train_policy_with_value(
        train_data=train_data,
        test_data=test_data,
        featurizer=featurizer,
        policy=policy,
        value_net=value_net,
        llm=llm,
        cfg=CFG,
    )
```

```
    print("Loading best saved policy/value for final evaluation...")
    if os.path.exists(CFG.best_policy_by_acc_path):
        print("Loading best RL policy by accuracy...")
        policy.load_state_dict(torch.load(CFG.best_policy_by_acc_path,␣
 ↪map_location=CFG.device))
    elif os.path.exists(CFG.best_policy_path):
        print("Loading best RL policy by reward...")
        policy.load_state_dict(torch.load(CFG.best_policy_path,␣
 ↪map_location=CFG.device))

    if os.path.exists(CFG.best_value_path):
        value_net.load_state_dict(torch.load(CFG.best_value_path,␣
 ↪map_location=CFG.device))

    print("\nFinal deterministic policy evaluation...")
    final_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
 ↪llm, CFG)
    print(json.dumps({"final_deterministic_eval": final_det}, indent=2))

    print("\nFinal sampled policy evaluation...")
    final_samp = evaluate_policy_sampled(test_data, featurizer, policy, llm,␣
 ↪CFG)
    print(json.dumps({"final_sampled_eval": final_samp}, indent=2))

    torch.save(policy.state_dict(), CFG.final_policy_path)
    torch.save(value_net.state_dict(), CFG.final_value_path)
    print(f"\nSaved final policy to {CFG.final_policy_path}")
    print(f"Saved final value net to {CFG.final_value_path}")


if __name__ == "__main__":
    main()
```

```
/opt/conda/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress
not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
torch version: 2.10.0+cu128
cuda available: True
cuda device count: 1
gpu name: NVIDIA H200 NVL

Clearing old cache at: ./cache_rl_two_stage_v9
Using device: cuda
Loading GSM8K…
Train subset: 300
```

```
Test subset : 100
Loading featurizer…

Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|      | 103/103 [00:00<00:00, 1022.54it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Loading policy and value network…
Loading LLM…
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[3], line 1199
   1195     print(f"Saved final value net to {CFG.final_value_path}")
   1198 if __name__ == "__main__":
-> 1199     main()

Cell In[3], line 1122, in main()
   1119     policy.mean_head.bias.fill_(0.45)
   1121 print("Loading LLM…")
-> 1122 llm = LLMReasoner(CFG.llm_name, CFG.device)
   1124 print_debug_examples(train_data, llm, CFG, budget=160, n=CFG.
 ↪debug_examples)
   1126 print("\nRunning fixed-budget baselines before warm-start/RL on TEST␣
 ↪set…\n")

Cell In[3], line 470, in LLMReasoner.__init__(self, model_name, device)
    466     self.tokenizer.pad_token = self.tokenizer.eos_token
    468 dtype = torch.float16 if device == "cuda" else torch.float32
--> 470 self.model = AutoModelForCausalLM.from_pretrained(
    471     model_name,
    472     dtype=dtype,
    473     device_map=       if device ==        else None
    474 )
    475 self.model.eval()
    476 self.device = device

File /opt/conda/lib/python3.13/site-packages/transformers/models/auto/
 ↪auto_factory.py:374, in _BaseAutoModelClass.from_pretrained(cls,␣
 ↪pretrained_model_name_or_path, *model_args, **kwargs)
```

```
    372     if model_class.config_class == config.sub_configs.get("text_config"␣
 ↪None):
    373         config = config.get_text_config()
--> 374     return model_class.from_pretrained(
    375␣
 ↪        pretrained_model_name_or_path, *model_args, config=config, **hub_kwar␣s, **kwargs
    376     )
    377 raise ValueError(
    378     f"Unrecognized configuration class {config.__class__} for this kind␣
 ↪of AutoModel: {cls.__name__}.\n"
    379     f"Model type should be one of {', '.join(c.__name__ for c in cls.
 ↪_model_mapping)}."
    380 )

File /opt/conda/lib/python3.13/site-packages/transformers/modeling_utils.py:
 ↪4001, in PreTrainedModel.from_pretrained(cls, pretrained_model_name_or_path,␣
 ↪config, cache_dir, ignore_mismatched_sizes, force_download, local_files_only,␣
 ↪token, revision, use_safetensors, weights_only, *model_args, **kwargs)
   3994     adapter_kwargs = {}
   3996 _adapter_model_path, pretrained_model_name_or_path, adapter_kwargs =␣
 ↪maybe_load_adapters(
   3997     pretrained_model_name_or_path,
   3998     download_kwargs_with_commit,
   3999     **adapter_kwargs,
   4000 )
-> 4001 device_map = check_and_set_device_map(device_map)  # warn, error and fi␣
 ↪the device map
   4003 user_agent = {"file_type": "model", "framework": "pytorch",␣
 ↪"from_auto_class": from_auto_class}
   4004 if from_pipeline is not None:

File /opt/conda/lib/python3.13/site-packages/transformers/integrations/
 ↪accelerate.py:134, in check_and_set_device_map(device_map)
    132         raise ValueError("DeepSpeed Zero-3 is not compatible with␣
 ↪passing a `device_map`.")
    133     if not is_accelerate_available():
--> 134         raise ValueError(
    135             "Using a `device_map`, `tp_plan`, `torch.device` context␣
 ↪manager or setting `torch.set_default_device(device)` "
    136             "requires `accelerate`. You can install it with `pip instal␣
 ↪accelerate`"
    137         )
    138 return device_map

ValueError: Using a `device_map`, `tp_plan`, `torch.device` context manager or␣
 ↪setting `torch.set_default_device(device)` requires `accelerate`. You can␣
 ↪install it with `pip install accelerate`
```

[ ]:
```
AI Disclouser: I have taken the help of ChatGPT while writing the code
```